Debreceni Egyetem
Informatikai Kar
Adattudomány és Vizualizáció Tanszék

Dinamikus objektumelkerülés megerősítéses tanulással a CARLA szimulátorban

Témavezető:
Mészáros László
Tanársegéd

Készítette:
Angi Dávid
Adattudomány MSc

Debrecen, 2026

Tartalomjegyzék

1. Bevezetés **4**

2. Szimulációs környezet **5**

3. Programozás és algoritmus **7**

3.1. Megerősítéses tanulás 8

3.2. Python programozási nyelv 9

3.3. Soft Actor-Critic és a Stable-Baselines3 10

4. Szenzoradatok előfeldolgozása, autoenkóderek **11**

4.1. Adatok 12

4.2. Kamera-autoenkóderek 13

4.2.1. Konvolúciós autoenkóder 14

4.2.2. Reziduális autoenkóder 15

4.2.3. VGG-alapú autoenkóder 15

4.2.4. Összehasonlítás és a választott modell 16

4.3. LiDAR-autoenkóderek 18

4.3.1. BEV-alapú autoenkóder 20

4.3.2. Gráf-alapú autoenkóder 21

4.3.3. Point-MAE 22

4.3.4. Összehasonlítás és a választott modell 23

4.4. Szenzorfúzió 24

5. Jutalomfüggvény **26**

5.1. R1: sávtartás és szabályos előzés 27

5.2. R2: ütközésidő-büntetés előzhető helyzetben 29

5.3. R3: hangolás bátrabb előzésre 29

5.4. R4: haladásalapú jutalom 30

5.5. R5: potenciálalapú jutalomformálás 31

6. Elkészült modellek összehasonlítása **32**

6.1. A kiértékelés módszertana 32

6.2. Eredmények 34

6.3. Modellek kiértékelése a szempontok alapján 34

6.3.1. Sebesség és menetidő 34

6.3.2. Előzések sikerességének vizsgálata 36

6.3.3. Kormányzás és sávtartás 39

6.3.4. Ütközések és visszahelyezések 40

7. Összegzés **41**

8. Köszönetnyilvánítás **42**

Nyilatkozat mesterséges intelligencia eszközök használatáról **43**

Hivatkozások **44**

1. Bevezetés

A 21. század egyik ha nem a legnagyobb hatású technológiai átalakulása a közlekedésben zajlik. Az elmúlt másfél évtizedben szemünk láttára kerültek önvezető járművek a kutatólaboratóriumokból a hétköznapi forgalom részévé. Kezdetben vezetéstámogató rendszerek, például a tempomat vagy későbbiekben a sávtartó igencsak kirívónak számítottak, azonban ma már a középkategóriás autókban is alapfelszereltség részét képezik, egyes városokban pedig sofőr nélküli robottaxik szállítják az utasokat. A fejlődés robbanásszerű gyorsulását a szenzorok relatíve olcsóbbá válása, valamint a járművekbe építhető, nagy teljesítményű számítási egységek megjelenése nélkülözhetetlen volt. Ezen felül az adatfeldolgozás és a mélytanulás fejlődése is kulcsszerepet ezen fejlődés történetben mivel, ezek a technológiák teszik lehetővé, a járművek számára környezetük érzékelését.

Az önvezető autók iránti érdeklődést elsősorban a biztonság motiválja. A közúti balesetekben világszerte évente több mint egymillió ember veszti életét [1], és a balesetek döntő többsége emberi hibára – figyelmetlenségre, fáradtságra, rossz helyzetmegítélésre – vezethető vissza.

Egy gép nem fárad el, semmi sem vonja el a figyelmét, valamint a reakcióideje optimális esetben rövidebb lehet az emberinél. Ez az eset azonban csak akkor valósulhat meg, ha a vezetést végző algoritmus minden helyzetben megbízhatóan, hatékonyan és biztonságosan működik. Nem elég, ha a jármű az esetek túlnyomó részében – egyenes úton, jó időben, kiszámítható hanem nehézkes körülmények között is helyt kell állnia. Ezen oknál fogva valódi kihívást a ritka, váratlan helyzetek jelentik, például a forgalom dinamikája, egy sávba kerülő akadály vagy egy előzés közben megváltozó forgalmi helyzet. Egyetlen rosszul kezelt helyzet is súlyos balesethez vezethet. Ezen megoldásokban a vezetés műveletét észlelésre, predikcióra, tervezésre, egyszóval jól elkülöníthető modulokra bontva oldják meg. Ezek a rendszerek jól értelmezhetők, de minden szabályt és határesetet előre meg kell fogalmazni bennük, ami a valós forgalom sokszínűsége mellett rendkívül nehéz. Alternatívát a megerősítéses tanulás (*reinforcement learning*, RL) kínál, amelyben az ágens nem előre megírt szabályokat követ, hanem a környezettel való interakció során, jutalmak és büntetések alapján maga tanulja meg, hogyan kell cselekednie. A megközelítés jól hasznosítható olyan döntési feladatoknál, mint a sávtartás, ahol a helyes viselkedést nehéz szabályokkal leírni, de könnyű megmondani, hogy egy adott kimenetel jó vagy rossz.

A vezetési feladatot hagyományosan egymásra épülő, alaposan tervezett modulokra bontják. Ezen blokkokat jól megtervezett logikai adatfolyamok segítségével kötik, ezeket a folyamokat robotikai rendszerek révén kötik össze ilyen például a Robot Operating System 2 (ROS2) [2]. Ilyen modulokra épülő önvezető rendszer például az Autoware Auto [3] amely egy ROS2 alapokra épülő robotikai rendszer önvezető autózáshoz. A rendszer kifinomultsága abban rejlik, hogy minden felhasználó saját igényei szerint tudja formálni a rendszert és saját modulokat is adhat hozzá. Az egyes modulok tehát cserélhetők, így például a szabályalapú döntéshozó modul helyettesíthető egy megerősítéses tanulás alapú ágenssel is. Hátránya viszont, hogy az ágens próbálkozások és

hibák sorozatán keresztül tanul, ami a valós világban igen költséges lenne, hiszen egy elfogadható vezetési stratégia kialakítása akár több százezer vagy több millió lépést is igényelhet. A tanulási folyamat során az ágensnek gyakorolnia kell és felfedeznie saját világát valamint korlátait. Ezt egy valós járművel elvégezni gyakorlatilag kivitelezhetetlen. Egy szenzorokkal felszerelt tesztjármű, a zárt tesztpálya, a biztonsági sofőr és a karbantartás költsége rendkívül magas, egy ütközés pedig nemcsak anyagi kárt okoz, hanem emberi életet is veszélyeztethet. Ráadásul az ágenst szeretnénk felkészíteni a váratlan és veszélyes helyzetekre, hogy azokat is megfelelőképp kezelje és elkerülje a baleseteket.

Ezen okoknál fogva a tanítást és a tesztelést szimulált környezetben végeztem. A szimulációra a CARLA-t [4] egy nyílt forráskódú, kifejezetten önvezetés-kutatásra fejlesztett szimulátor, amely valósághű városi és autópálya-környezetet is képes egyaránt szimulálni valamint a fizikai járműmodellt, amelyek elengedhetetlenek a reális valósághű szimulációkhoz.

Egy önvezető rendszer döntései csak annyira lehetnek jók, amennyire pontos a környezetéről alkotott képe, ez a kép pedig kizárólag a gépi látás és érzékelés révén a szenzorokból származik. A szenzoradatok feldolgozása az ágens számára igazi kihívást jelent. A komplex szenzorokból sokszínű és nagy varianciájú adat érkezik. Ezért munkám során a kamera és a LiDAR szenzorokból érkező nyers adatokat előfeldolgoztam, majd szenzorfúzió segítségével egyesítettem ezeket. Az ágens egy tömör, így már mindkét szenzor információját tartalmazó bemenetet kap a külvilág aktuális állapotáról.

Munkám során olyan ágens kifejlesztésén szeretnék dolgozni amely képes forgalmi szituációkat megfelelőképp biztonságosan kezelni például egy az ágensnél lassabban közlekedő forgalmi szereplő elkerülése. Az alapötletet Terapaptommakol és munkatársai [5] munkája adta, akik a CARLA szimulátorban DQN alapú ágenst tanítottak, amely a LiDAR pontfelhő, a pozíció és a sebesség alapján fékezéssel és kitéréssel sikeresen elkerülte az úton parkoló járművet. Ez a munka azonban kizárólag statikus akadályra terjedt ki. Munkám során a fent említett kutatást egészítem ki két irányban. Először is a statikus akadályok helyett mozgó az ágensnél lassabban közlekedő járművek biztonságos elkerülését betartva a KRESZ szabályait. Másodlagosan a LiDAR mellett a kamerát is bevontam az érzékelésbe szenzorfúzióval.

2. Szimulációs környezet

A tanításhoz és a teszteléshez a CARLA (*Car Learning to Act*) szimulátorra [4, 6] esett a választásom (1. ábra). A CARLA egy nyílt forráskódú, Unreal Engine 4 (UE4) alapú szimulációs környezet, amelyet kifejezetten önvezető rendszerek fejlesztésére, tanítására, validálására és tesztelésére hoztak létre. A szimulátor magja C++ nyelven készült, a megjelenítést és a fizikai szimulációt pedig az UE4 játékmotor végzi. Ennek köszönhetően a szimulátor valósághű képet és járműdinamikát nyújt. A program képes a valóságban is gyakran használt önvezető szenzorok – többek között RGB-kamera, mélységkamera, szemantikus szegmentációs kamera, LiDAR, radar,

GNSS és IMU – szimulált megfelelőit generálni minden forgalmi szituációban. Továbbá fel van készítve a dinamikus időjárás, napszak realisztikus kezelésére is. A forgalom pedig szabadon állítható és szabályozható. Így az ágens olyan típusú szenzoradatokból tanulhat, mint amilyeneket egy valódi jármű is szolgáltatna.

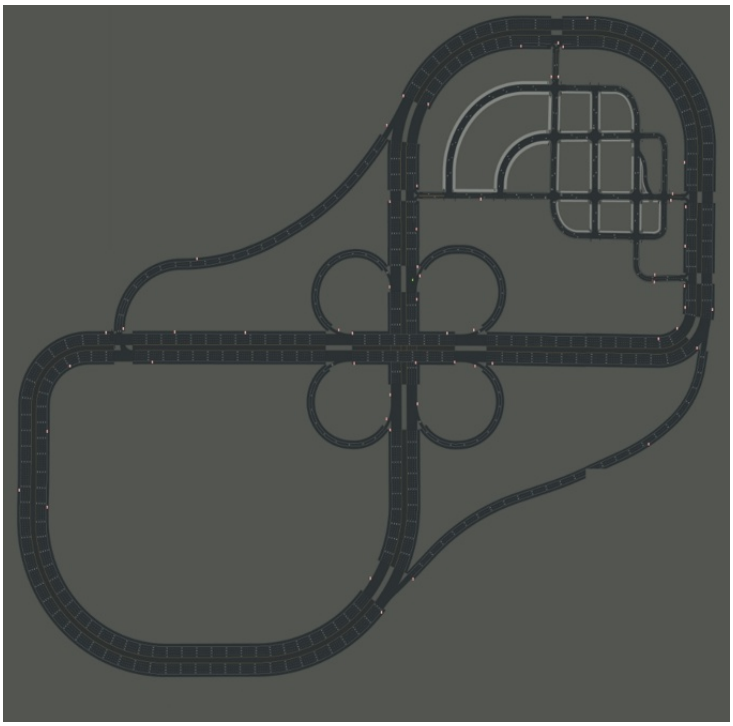


1. ábra. A CARLA szimulátor (forrás: [6])

A szimulátor kliens–szerver architektúrára épül [7], tehát a szerver futtatja magát a szimulációs környezetet, ezáltal ő felel a világ és a járművek fizikájának számításáért, valamint a szenzoradatok előállításáért. Ez a kialakítás ugyanakkor teret enged annak is, hogy a munka akár két számítógépen is megvalósuljon, például ez akkor indokolt, ha nem elegendő egy számítógép grafikus feldolgozási erőforrása arra, hogy a szimuláció és az ágens tanulása is fusson egyaránt. A kliens program, melyet a felhasználó ír, hálózati interfészen, TCP-kapcsolat segítségével éri el a szimulátort. A kommunikáció alapértelmezetten a 2000-es porton működik, illetve ugyanitt küldi a parancsokat a szervernek. Ilyenek lehetnek például a jármű kormányparancsa, gázadagolása és fékezése is, ugyanakkor a kliens így kapja meg az egyes szimulálni kívánt szenzoradatokat is, a megfigyelni kívánt járműről. A kliensoldali API Pythonban is elérhető, ami nagyban megkönnyíti a fejlesztést. Munkám során a CARLA 0.9.16-os verzióját használtam szinkron módban, rögzített 20 Hz-es szimulációs lépésközzel. Szinkron módban a szerver csak akkor lép a következő állapotba, amikor a kliens erre utasítja. Ez nagy segítségemre volt az ágens egyes lépéseinél, így a szenzoradatok és az ágens döntései mindig ugyanahhoz a szimulációs lépéshez tartoztak, valamint a tanítás is reprodukálható marad.

A CARLA-ban adott a lehetőség útvonaltervezésre is. A pálya úthálózata útpontokból (*waypoint*) álló topológiai gráfként érhető el, amelyen a szimulátorhoz tartozó `GlobalRoutePlanner` segítségével két tetszőleges pont között játszi könnyedséggel tervezhető [7] bármilyen kívánt

útvonal. Ezt kihasználva minden epizód elején véletlenszerűen választott kezdő- és célpont között új, dinamikus útvonalat tervezek, amelyen az ágensnek végig kell haladnia, ezzel kiküszöbölve azt, hogy esetlegesen rátanuljon az adott útvonalra, így segítem az általánosítási képességét.



2. ábra. A Town04 pálya felülnézeti térképe (forrás: [7])

A szimulátor alapértelmezetten tartalmaz több előre elkészített pályát (*Town*), amelyek variánciája az átlagos önvezető feladatok többségét kielégíti. Munkám során ezek közül a Town04-et választottam (2. ábra). Választásom erre a pályára esett, mivel az itt kialakított kisváros tökéletesen megfelel az elvárásaimnak. Alakja „8-as" alakú, végtelenített autópályával [7] megfelelő célom eléréséhez. A pálya egyszerre tartalmaz városi utcákat és többsávos autópályát, fel- és lehajtókkal, valamint lóhere alakú csomóponttal. Amint említettem, célom az autópályás előzés megtanítása, ez a pálya a legalkalmasabb. A többsávos autópályán az ágens a lassabban haladó járművet a bal oldali sávba kihúzódva szabályosan meg tudja előzni, a körbeérő autópálya pedig hosszú, megszakítás nélküli vezetési szakaszokat tesz lehetővé.

3. Programozás és algoritmus

A következőkben munkám elméleti és technikai hátterét írom le röviden. Elsőként a megerősítéses tanulás alapjait, majd ezt követően a fejlesztéshez használt programozási nyelvet és könyvtárait, végül a Soft Actor-Critic algoritmust és a Stable-Baselines3 keretrendszert. Mindegyiknél kitérve arra, miért esett az adott technológiára a választásom.

3.1. Megerősítéses tanulás

A gépi tanulás módszereit hagyományosan három nagy csoportba soroljuk. A *felügyelt tanulás* során a modell tisztában az elvárt kimenet valamint a bemeneti adatokkal egyaránt így tanul, vagyis minden tanítópéldához ismert a helyes válasz. Ide tartoznak a klasszikus osztályozási és regressziós feladatok, például képeken látható objektumok felismerése, de maga a vezetés is megtanulható így. A *viselkedésklónozás* (*behavior cloning*) során a modell egy emberi sofőr rögzített vezetéséből, a szenzoradatok és a hozzájuk tartozó kormány valamint gázparancsok párjaiból tanulja meg utánozni a sofőrt. Ezt az elvet már az 1980-as évek végén alkalmazta az ALVINN rendszer [8]. A módszer korlátja, hogy a modell legfeljebb olyan jó lehet, mint a tanítóadat. Értem ez alatt, hogy a modell nagy varianciájú adaton kell tanuljon különben a váratlan helyzeteket nem tudja megfelelőképp kezelni. Általánosítási képessége kevésbé erős, mint a többi gépi tanulási megoldásnak. Következő módszer *felügyelet nélküli tanulás* ahol nincsenek címkék, a tanítás célja az adatok rejtett szerkezetének, például csoportjainak vagy tömör reprezentációjának feltárása. Tipikus példája a klaszterezés például a k-közép módszer, amely a hasonló adatpontokat csoportokba rendezi.

A *megerősítéses tanulás* ezektől alapvetően eltér. A tanuló ágensnek nem adjuk meg előre, mi lett volna a helyes döntés egy adott helyzetben. Ehelyett az eljárás értelmében, jutalomfüggvénnyel értékeljük viselkedését. Ha a leírtaknak megfelelően viselkedik jutalmazzuk, a szabályokat ha megszegi büntetjük, ebből a számszerű visszajelzésből tanulja meg a helyes viselkedést [9]. A tanulási folyamatnak két szereplője van egyik az *ágens*, a döntések meghozataláért felelős a *környezet*, amelyben az ágens mozog a folyamat során. Ezen páros kölcsönhatása egymással diszkrét időlépésekben történik: az ágens minden lépésben megfigyeli a környezet aktuális *állapotát*, ez alapján végrehajt egy *akciót*, amelynek hatására a környezet új állapotba kerül, az ágens pedig jutalmat kap előző döntései alapján. A feladatot Markov-döntési folyamatként (*Markov Decision Process*, MDP) írhatjuk fel [9], amiben az állapotok, az akciók, és az állapotátmenet-valószínűségek valamint a jutalomfüggvény határoz meg. A fentiek alapján a modell feltevése a *Markov-tulajdonság*. A következő állapot csak a jelenlegi állapottól és akciótól függ. Mivel az ágens a világot szenzorain keresztül érzékeli, ezt tekintjük állapotnak.

Az ágens viselkedését a *stratégia* (*policy*) írja le, célja minden tanulási epizódban a jutalmakból képzett *hozam* várható értékének maximalizálása:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \tag{1}$$

ahol r_{t+1} a t-edik lépés utáni jutalom, a $\gamma \in [0,1)$ diszkontfaktor pedig a jövőbeli jutalmak súlyát szabályozza. A tanulás *epizódokra* tagolódik, amelyek nálam ütközéssel vagy az útvonal teljesítésével érnek véget.

Nagy dimenziójú és folytonos állapotterek esetén – amilyeneket a szenzoralapú kameraképek

és a LiDAR pontfelhők is reprezentálnak – az értékfüggvény, illetve a döntési stratégia megfelelő közelítésére hatékony módszer a mély neurális hálózatok használata (*deep reinforcement learning*). Én is ezt a megközelítést alkalmaztam. Ezen típusú algoritmusok között az áttörést a DQN hozta [10], amely *visszajátszási pufferrel* (*replay buffer*) és *célhálóval* (*target network*) stabilizálta a tanulást.

Az algoritmusválasztás szempontjából a fő különbségek [9]:

- **Modellalapú vs. modellmentes:** a CARLA fizikájának megtanulása nehéz lenne, ezért modellmentes módszert választottam.
- **Értékalapú, stratégiaalapú, aktor–kritikus:** utóbbiban az *aktor* a stratégia, a *kritikus* pedig értékeli a döntéseit.
- **On-policy vs. off-policy:** az on-policy módszerek eldobják a régi tapasztalatot, az off-policy módszerek (DDPG [11], TD3, SAC [12]) újrahasznosítják, így kevesebb interakcióból tanulnak.
- **Diszkrét vs. folytonos akciótér:** a DQN csak véges akciók közül választhat.

3.2. Python programozási nyelv

A kísérletem implementálására a Python programozási nyelvet választottam, amely magas szintű, interpretált, dinamikusan típusos valamint általános célú. Megálmodója Guido van Rossum, a nyelv első változata 1991-ben jelent meg [13]. Tervezési filozófiájának középpontjában az olvashatóság áll. Ezen elvre kiváló példa, hogy a kódblokkokat behúzás jelöli, a szintaxis tömör és közel áll a természetes nyelvhez, így a programok gyorsan megírhatók és könnyen érthetők. Ezen tulajdonságainak köszönheti, hogy ma a világ egyik legszélesebb körben használt programozási nyelve, a gépi tanulás és az adattudomány területén pedig gyakorlatilag szabványnak számít.

Elsőre ellentmondásosnak tűnik, hogy egy rendkívül számításigényes gépi tanulási problémát egy viszonylag lassú nyelvben történő megvalósítás követ, de a magyarázat igen logikus. A Python ezen a területen „ragasztónyelvként” működik. Értem ez alatt, hogy a nagy számítási igényű műveleteket, a mátrixszorzás, konvolúció vagy a gradiensszámítás, C, C++ és CUDA nyelven írt, optimalizált könyvtárak végzik a processzoron vagy a grafikus kártyán. A Python pedig ezen optimalizált könyvtárak kényelmes, magas szintű használatát biztosítja. Az interpreter által okozott műveletek többletideje elhanyagolható, köszönhetően ezen könyvtárak natív implementációjának. A fejlesztő így egy könnyen írható nyelv kényelmét élvezi, a számítások mégis natív sebességgel futnak.

A neurális hálókat PyTorch [14] keretrendszerben valósítottam meg. Ez GPU-gyorsított tenzorműveleteket és automatikus differenciálást biztosít, a számítási gráfot pedig futás közben építi fel, ami megkönnyíti a fejlesztést és a hibakeresést. A választást az is indokolta, hogy

a Stable-Baselines3 is PyTorch-ra épül, így a későbbi neurális hálók és az ágens hálói egy keretrendszerben, közös GPU-n futhatnak.

A Python mellett szólt végül, de nem utolsósorban, hogy a CARLA kliensoldali API-ja is elérhető Pythonban (2. fejezet), ámbár a kliens maga eredetileg C++ nyelven íródott. Így egyetlen nyelven valósulhatott meg a szimulátor vezérlése, a szenzoradatok feldolgozása, a neurális hálók tanítása és maga a megerősítéses tanulás is.

3.3. Soft Actor-Critic és a Stable-Baselines3

Kísérleteim alapjául a Soft Actor-Critic (SAC) algoritmust választottam. A választott algoritmus egy off-policy aktor–kritikus módszer, amely folytonos akciótérben sztochasztikus stratégiát tanul. Az eljárás minden állapothoz párosít egy valószínűségi eloszlást majd az akciók halmazából. Később ebből mintavételezi a végrehajtandó akciót.

A célja nemcsak a jutalom, hanem a stratégia entrópiájának maximalizálása is:

$$J(\pi) = \sum_t \mathrm{E}\Big[\, \gamma^t \Big(r(s_t, a_t) + \alpha\, \mathscr{H}\big(\pi(\cdot \mid s_t)\big) \Big) \Big], \tag{2}$$

ahol s_t a t-edik időlépés állapota, a_t az ekkor választott akció, $r(s_t, a_t)$ az érte kapott jutalom, $\pi(\cdot \mid s_t)$ a stratégia által adott akcióeloszlás, $\mathscr{H}$ ennek entrópiája, amely azt méri, mennyire szétterült az akcióeloszlás, α pedig a *hőmérséklet*, amely az entrópia súlyát adja meg a jutalomhoz képest. A γ diszkontfaktor, amelynek értéke nálam 0,98, a későbbi jutalmakat kisebb súllyal veszi figyelembe. A várható érték a stratégia követésével bejárt állapotokra és akciókra vonatkozik.

A fentiekben elmondottak alapján az ágens addig marad változatos, amíg ez nem rontja érdemben a jutalmat. Így a felfedezés nagyrészt magából a célfüggvényből következik: az ágens kevésbé hajlamos korán leragadni egy rossz megoldásnál, és nem kötelezi el magát idő előtt egyetlen viselkedés mellett, például abban, hogy most előzzön-e vagy kicsit később. A Stable-Baselines3 (SB3) megvalósításában a stratégia egy Gauss-eloszlás, amelynek mintáit a tanh függvény szorítja a megengedett akciótartományba. A felfedezést nálam ezen felül az állapotfüggő felfedezés (gSDE) [15] segíti: a zaj az aktuális állapottól függ, és nem minden lépésben sorsolódik újra, hanem csak 8 lépésenként, így a kormány- és gázmozdulatok simábbak, nem remegnek lépésről lépésre.

A tanítást két kritikus, vagyis két Q-háló segíti, ezek alapértelmezetten implementálva vannak az SB3-ban. Ezek becslik a Q-függvényt, két kritikus egy közös jellemzőkinyerő hálón osztozik a szenzoradatokat feldolgozó csak a rá épül külön rétegként. A tanítás célértékét a két becslés kisebbikéből számolja, ami csökkenti a Q-értékek túlbecslését. A tanítás a visszajátszási pufferből vett kötegeken történik. Az α értékét az algoritmus automatikusan hangolja [16]: úgy állítja, hogy a stratégia entrópiája egy előre adott célérték közelében maradjon. Az SB3-ban ez az akciótér dimenziójának mínusz egyszerese, nálam -2.

Az algoritmust nem magam implementáltam, hanem a Python SB3 [17] könyvtárát használtam könnyű használhatósága valamint népszerűsége okán. Az SB3 egy megfelelően kitesztelt PyTorch-alapú megvalósítás, emiatt a munkám a hatékony környezeti jellemzők kinyerésére és a megfelelően finomhangolt jutalomra összpontosulhatott. Az SB3 a Gymnasium [18] szabványú környezetekkel működik, amelyek a `reset` és `step` függvényeken keresztül indítanak epizódot és hajtanak végre akciót. A szabvány megkülönbözteti a valódi végállapotot (*terminated*) az olyan megszakítástól (*truncated*), amely után a vezetés folytatható lenne; utóbbinál az SB3 a következő állapot becsült értékével számol tovább. Nálam valódi végállapot az ütközés, a pályaelhagyás, a megállás és a túl gyors haladás, megszakítás pedig az útvonal végének vagy a kitűzött távolságnak az elérése.

Megvalósításaim:

- a CARLA-környezet Gymnasium interfésszel, amely a járművet, a szenzorokat, az útvonalakat, a forgalmat és az epizódokat kezeli;
- a megfigyelések előállítása a szenzoradatokból (4. fejezet);
- a jutalomfüggvény és az epizódvég feltételei (5. fejezet);
- a kamera- és LiDAR-jellemzőket összegző, majd a járműadatokkal és az útvonal jellemzőivel összefűző háló;
- a fontosabb mutatókat TensorBoardba naplózó callbackek.

4. Szenzoradatok előfeldolgozása, autoenkóderek

A robotikai eszközök, például az autók, környezetüket rájuk erősített szenzorok segítségével érzékelik. Ezen eszközök halmazának segítségével valósítják meg a gépi látás és érzékelés folyamatát. Míg az emberek saját érzékszerveikkel jutnak információhoz a külvilágról, a robotikai eszközök, szenzorrendszereikkel végzik el ezt az igen kritikus lépést a tájékozódás során. A szenzoradatok a pontos érzékelés okánál fogva nagyon sokrétűek és sűrűek. Emiatt nagy számítási kapacitást igényel feldolgozásuk során valamint általában átesnek egy adattisztítási folyamaton is. Az említett folyamatok szükségessége abban nyilvánul meg, hogy az ágens önmagában nem képes feldolgozni ezt a rengeteg információt, valamint ha mégis, ezek a feldolgozási folyamatok nagyban lassíthatják a döntés megszületését.

Ebben a fejezetben bemutatásra kerülnek az általam használt szenzorok, valamint az is, hogy miként végeztem el az előfeldolgozási folyamatokat. Ágensem érzékeléséért két fő szenzor felelős, egy LiDAR valamint egy RGB kamera.

Szenzorok választásánál a kiinduló munka [5] kizárólag LiDAR-ra támaszkodott. Ehhez hozzáadtam az RGB kamera információit. Esetemben a két szenzor kiegészíti egymást, a LiDAR

pontos térbeli geometriát ad például milyen hosszú az éppen elkerülendő jármű. Ezt az érzékelést hivatott kiegészíteni a kamera ahol a színes kép információkal szolgál például a sávfelfestésekről, valamint képes előrébb érzékelni, ha van az ágens előtt valaki. Az önvezető rendszereknek, hogy mely szenzorok elegendők, a szakirodalomban és az iparban sincs egyetértés. Léteznek kizárólag kamerára épülő megoldások [19], ugyanakkor mások a LiDAR mellett további szenzorokat is alkalmaznak.

A kamera a vizuális észlelés, hogy lesz előttem egy jármű, valamint azért felelős, hogy az ágens pontosan lássa a sávok felfestését az útburkolaton. A LiDAR szenzoradatok a térbeli letapogatásért felelős, információt szolgáltat például a megelőzendő jármű fizikai dimenzióiról. Akkor lehet hasznos, hogy ne próbálja túl korán befejezni az előzési manővert.

A CARLA mindkét szenzort valósághűen szimulálja [7]. A kamera képét az UE4 grafikus motorja rendereli, és a valódi kamerákra jellemző hatásokat is utánozza: automatikus expozíciót, mozgási elmosódást, becsillanást, mélységélességet stb. A LiDAR-t sugárkövetéssel szimulálja: a csatornák száma, a hatótávolság, a látószög és a forgási frekvencia a valódi eszközökhöz hasonlóan beállítható, a mérésekhez zaj adható, és a pontok egy része véletlenszerűen elveszik, ahogy a valóságban is.

Előzetes tapasztalataim alapján nem optimális a nyers szenzoradatokat közvetlenül a modellnek adni. Egyetlen kamerakép is több tízezer számértékből ami igen megterheli a modellt futását a LiDAR pedig több tízezer pontból áll, ráadásul a pontok száma és sorrendje lépésről lépésre változik. Ha az ágens ezeket nyersen kapná, a jutalomjelből kellene megtanulnia kinyerni a szenzorokból a lényeges információkat és vezetni. Ezek mellett a nagy bemenet a visszajátszási puffer memóriaigényét is jelentősen megnövelné. Így elkezdtem tömöríteni a szenzoradatokat ehhez beágyazásokat készítettem, amelyekhez autoenkódereket alkalmaztam. A kiinduló munkában [5] a pontfelhőt kézzel egyetlen jellemzőre szűkítették, a jármű előtti sávban lévő legközelebbi pont távolságára, így a háló bemenete mindössze hat érték volt. Ezzel szemben az autoenkóder által tanult reprezentáció a teljes jelenet lényegi információját megőrzi, így az ágens a mozgó járművek helyzetét és a sávokat is számon tudja tartani. Az autoenkóder egy kódolóból és egy dekódolóból álló háló, amely úgy tanul, hogy a bemenetét egy kis méretű belső reprezentáción, az úgynevezett látens téren keresztül próbálja visszaállítani. Az így kinyert látens vektor vagy címke már csak az adatok lényegi információit tartalmazza. A tanítás után a kódoló kimenete adja az ágensnek a bemenetét, így a tanulás elegendő ha a vezetés elsajátítására összpontosulhat.

4.1. Adatok

Az autoenkóderek tanításához saját adatgyűjtő programot írtam. A program a Town04 pályán, 200 forgalmi jármű között vezeti az autót, és ugyanazokkal a szenzorbeállításokkal rögzít, mint amelyeket az ágens a tanulás során lát. Mivel a CARLA autopilotja mindig pontosan a sáv közepén halad, az autót a sávon belül folyamatosan jobbra-balra térítettem, és időnként véletlen

kormánymozdulatokat is adtam neki. Így az adatban a sávból kisodródott, ferdén álló helyzetek is szerepelnek, ami növeli az adatok varianciáját. A szimuláció 20 Hz-en fut, de csak minden negyedik lépést mentettem, vagyis 5 Hz-cel, mivel az egymást közvetlenül követő képek szinte azonosak, és fölöslegesen növelnék az adat méretét.

A tanítások során összesen 40 211 egymáshoz tartozó kamera–LiDAR adatpárt gyűjtöttem. Az összetartozó kép, pontfelhő párok fájlneve a szimulációs képkocka sorszáma, így biztosítottam, hogy ugyanahhoz a pillanathoz tartozzanak. A kameraképet a szimulátor 640×320 pixeles felbontásban rendereli, ezt mentés előtt 160×80 pixelesre kicsinyítettem, és 8 bites RGB PNG-képként tároltam. A LiDAR esetén egy 64 csatornás modellt használtam, amely egy körülfordulás alatt 64×1024 sugarat bocsát ki. Ebből mintegy 30 ezer pont marad, amelyeket ritkítás nélkül, XYZ koordinátáikkal, méterben, 32 bites lebegőpontos számként mentettem. Az adatokat véletlenszerű keverés után 70/15/15 arányban osztottam fel, így 28 149 képkocka került a tanító-, 6 031 a validációs és 6 031 a teszthalmazba. A keverésre azért volt szükség, mert az egymást követő képkockák szinte azonosak, és enélkül a halmazok a pálya más-más szakaszait tartalmaznák.

4.2. Kamera-autoenkóderek

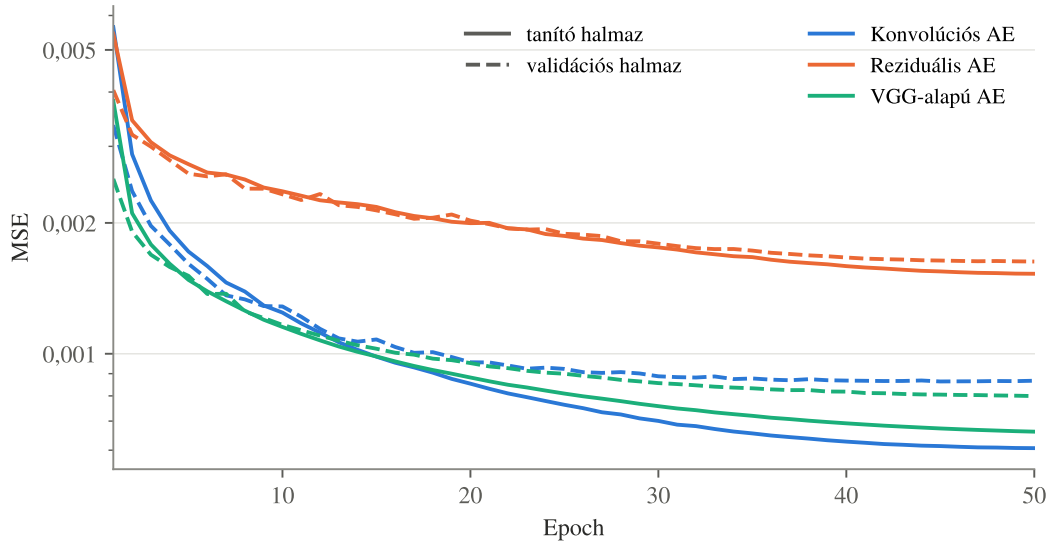
Az ágens kamerája a CARLA szimulált `sensor.camera.rgb` RGB kamerája, amelynek a helyzete a járművön a szélvédő mögött van. Az autó testének középpontjához képest 1,6 m-rel előrébb és 1,7 m magasan, menetirányba néz.

A képet 160×80 pixeles felbontásban, 110°-os vízszintes látószöggel, a szimulációval szinkronban 20 Hz-en rögzíti (3. ábra). Ez a relatíve kisebb felbontás tökéletesen elegendő a sávjelek valamint a forgalom többi szereplőjének felismeréséhez.



3. ábra. Példák az ágens által látott 160×80 kameraképekből

Miután kész voltam a tanító adatok begyűjtésével, kiválasztottam három különböző architektúrára épülő autoenkódert, hogy ezekből kiválasszam a számomra legjobbat. A három modellt egyenlő feltételek mellett, ugyanazon a rekonstrukciós feladaton tanítottam. Hibafüggvényként a bemeneti és a visszaállított kép közötti átlagos négyzetes hibát (MSE) minimalizáltam Adam optimalizálóval. Mindhárom modellt 50 epochon át, 64-es kötegmérettel tanítottam, a tanulási ráta koszinuszos ütemezéssel 10^{-3}-ról 10^{-5}-re csökkent, a VGG-alapú modellnél 10^{-4}-ről indulva. A modellek bemenete a 160×80 pixeles RGB kép, amelynek pixelértékeit a $[0,1]$ tartományba normalizáltam. A hiba alakulását a 4. ábra mutatja: a validációs hiba egyik modellnél sem kezdett nőni, vagyis érdemi túltanulás nem lépett fel.



4. ábra. A kamera-autoenkóderek tanító és validációs hibája epochonként, logaritmikus skálán

Az eljárás során a validációs halmazon legjobbnak bizonyult modellt mentettem el. Az alábbi ábrák mindegyike ugyanazt a négy, a tanítás során félretett teszthalmazból származó képet mutatja, ez által a modellek rekonstrukciói közvetlenül összevethetők.

4.2.1. Konvolúciós autoenkóder

Első választottam egy egyszerű konvolúció autoenkóderre esett amely szerkezetileg a következő. A kódoló négy konvolúciós blokkból áll. Ezek 4×4-es kernellel és 2-es lépésközzel dolgoznak, batch-normalizálást és LeakyReLU aktivációt használnak, és mindegyik felezi a kép méretét, ezek által $3\times80\times160$-as bemenetből $256\times5\times10$-es jellemzőtérkép lesz. Ez egy teljesen összekötött réteg amely vetítésre kerül a látens vektorba a kinyert jellemzőkkel. A dekódoló ennek tükörképe transzponált konvolúciókkal, a kimenetet egy szigmoid függvény adja és szorítja a $[0,1]$-es tartományba. Egyszerű, sekély felépítése miatt a három modell közül ez a leggyorsabb. A

rekonstrukción (5. ábra) jól kivehetők a sávjelek, az útnak a széle és a járművek egyaránt jól kivehetők. A háttér apró részletei mosódnak el de az jelen esetben amúgyis zajnak számít.



5. ábra. A konvolúciós autoenkóder rekonstrukciói a teszthalmazon

4.2.2. Reziduális autoenkóder

Második autoenkóder jelöltem egy Reziduális blokkokra épülő modell. Ebben a rétegek kimenetéhez hozzáadódik a bemenetük $(x + F(x))$; ez mélyebb hálóknál is stabil tanulást tesz lehetővé [20]. A háló négy felbontási szinten dolgozik, szintenként két reziduális blokkal, és többfelbontású átkötéseket is felhasznál. Minden szint kimenete közvetlenül a szűk keresztmetszet elé is eljut, hogy ez által a finom részletek is bekerülhessenek a látens reprezentációba. A három modell közül ennek van a legkevesebb, 2,4 millió paramétere. A rekonstrukció (6. ábra) erősen elmosott: az út nagyjából felismerhető, de a jármű körvonala szétfolyik, a környezet pedig általános, ködös foltokká válik.



6. ábra. A reziduális autoenkóder rekonstrukciói a teszthalmazon

4.2.3. VGG-alapú autoenkóder

A harmadik megvalósítás egy a VGG-19 hálózat [21] első négy blokkja a `relu4_1` rétegig, az ImageNet adathalmazon előtanított súlyokkal. Előtanítottsága okán az alapvető képi jellemzők, mint az élek és a textúrák felismerését a hálónak nem kell nulláról megtanulnia. A dekódoló a kódoló tükörképe. Mivel ez egy mély, normalizálás nélküli háló, kisebb, 10^{-4}-es tanulási

rátával kellett tanítani. Érdekesség, hogy az előtanított súlyok nélkül egyáltalán nem tanult. A rekonstrukción (7. ábra) a sávjelek és a jármű megmaradnak, de elmosódnak, és a modell helyenként a jelenetre jellemző, de valójában ott nem lévő elemeket rajzol. A második képen az épület és a téglafal helyén fákat rajzol, amelyek valójában ott sincsenek.



7. ábra. A VGG-alapú autoenkóder rekonstrukciói a teszthalmazon

4.2.4. Összehasonlítás és a választott modell

A modelleket a 40 211 képből félretett, 6031 képes teszthalmazon hasonlítottam össze. Az MSE mellett a csúcsjel-zaj viszonyt (PSNR) is megfigyeltem, amely a $[0,1]$ tartományba skálázott képekre a következő:

$$\mathrm{PSNR} = 10 \log_{10} \frac{1}{\mathrm{MSE}} \ \mathrm{dB}, \tag{3}$$

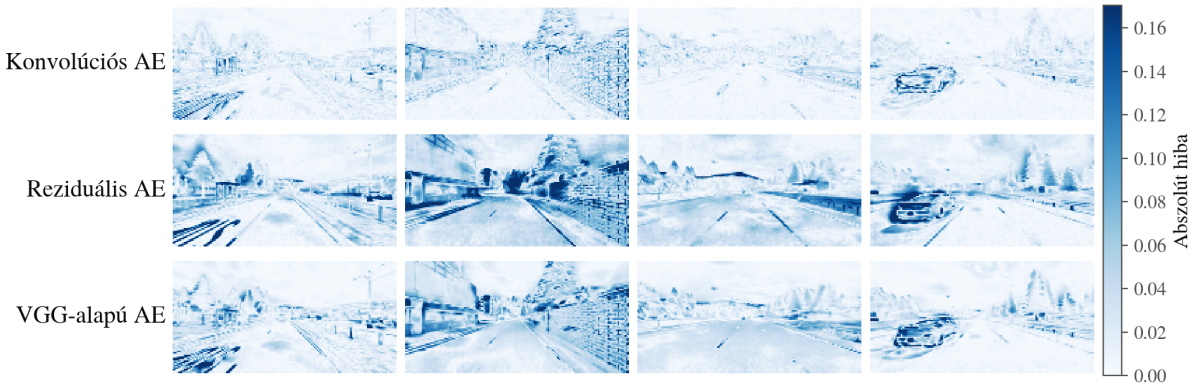
Ezen mérőszám minél nagyobb, annál pontosabb a visszaállított kép. A kódoló az ágens tanítása közben minden lépésben, ugyanazon a GPU-n fut így a kódolási idő is szignifikáns jellemzők közé soroltam (1. táblázat).

1. táblázat. A három kamera-autoenkóder összehasonlítása a teszthalmazon

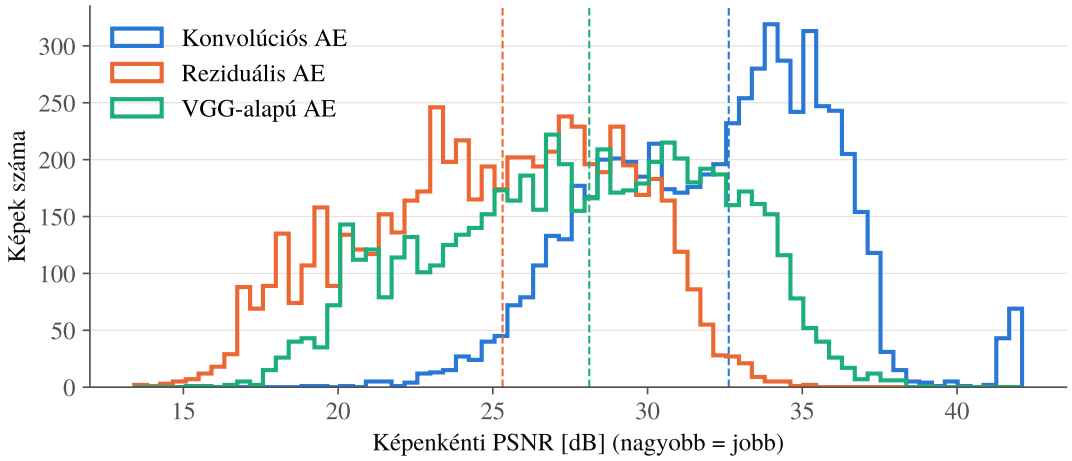
Modell	MSE	PSNR	Paraméter	Kódolás
Konvolúciós AE	**0,00088**	**30,58 dB**	4,5 M	**0,27 ms**
Reziduális AE	0,00495	23,06 dB	**2,4 M**	0,93 ms
VGG-alapú AE	0,00278	25,56 dB	8,7 M	1,01 ms

A fenti táblázat alapján a konvolúciós autoenkóder hibája a legkisebb. Az MSE-je harmada a VGG-alapú megoldáshoz képest és kevesebb, mint ötöde a reziduális modellének, ami 5, illetve 7,5 dB-lel jobb PSNR-t jelent. A különbség nemcsak átlagban áll fenn, a képenkénti párosítás során kiderült, hogy a konvolúciós megoldás a teszthalmaz képeinek 97,4%-án kisebb a hibája, mint a VGG-alapú modellé, a reziduális modellnél pedig minden képen jobb (9. ábra).

A hibatérképeken (8. ábra) az is látszik, hogy a konvolúciós modell rekonstrukciója a legkevésbé zajos. A hiba nála csak a tárgyak éleinél jelenik meg vékony vonalakként. A többi modell esetében nagyobb, összefüggő foltokban, például a jármű és a sávjelek környékén, vagyis ott, ahol az ágensnek a legpontosabb információra volna szüksége.



8. ábra. A rekonstrukciók abszolút hibája képpontonként, közös skálán; a sötétebb szín nagyobb hibát jelöl



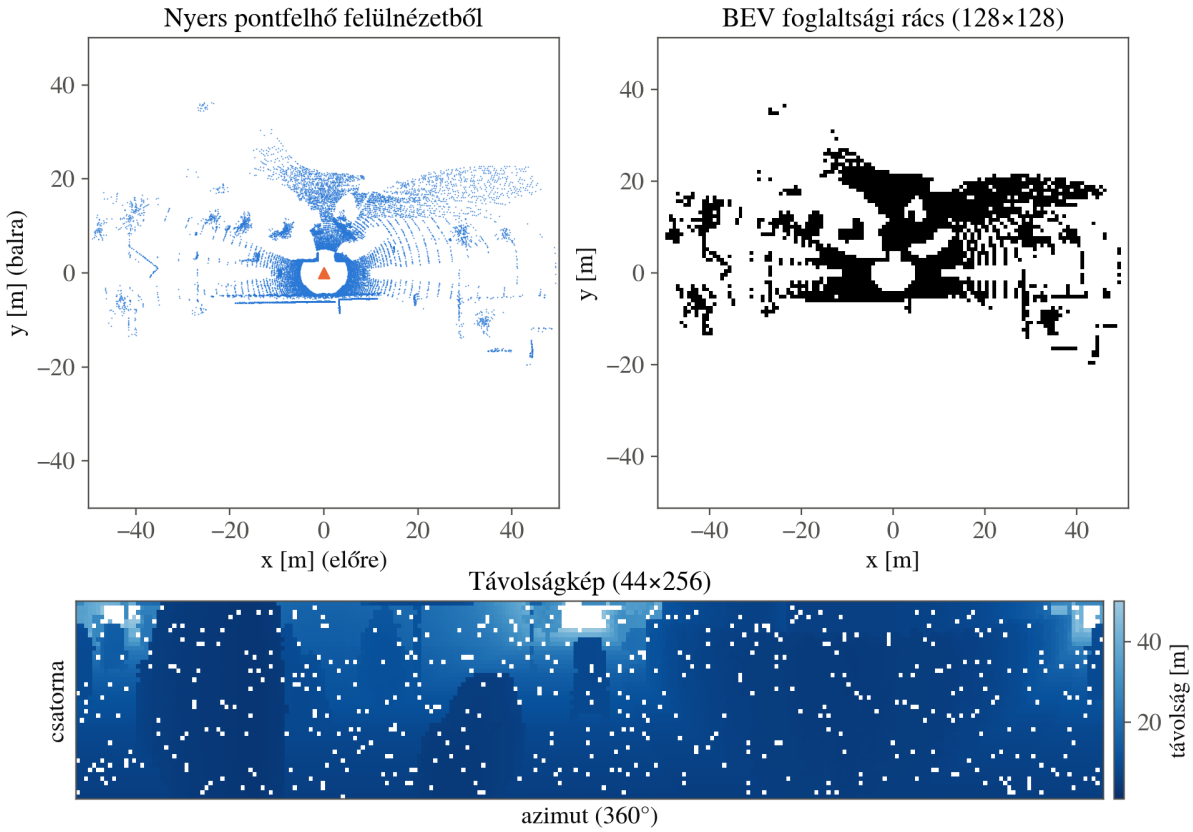
9. ábra. A képenkénti PSNR eloszlása a teszthalmazon; a szaggatott vonalak a modellek mediánját jelölik

Ezen okoknál fogva választásom a konvolúciós autoenkóderre esett. A legjobb rekonstrukció mellett ez a leggyorsabb is. Egy kép kódolása kb. 0,27 ms, a másik két modellnél ez közel négyszer annyi.

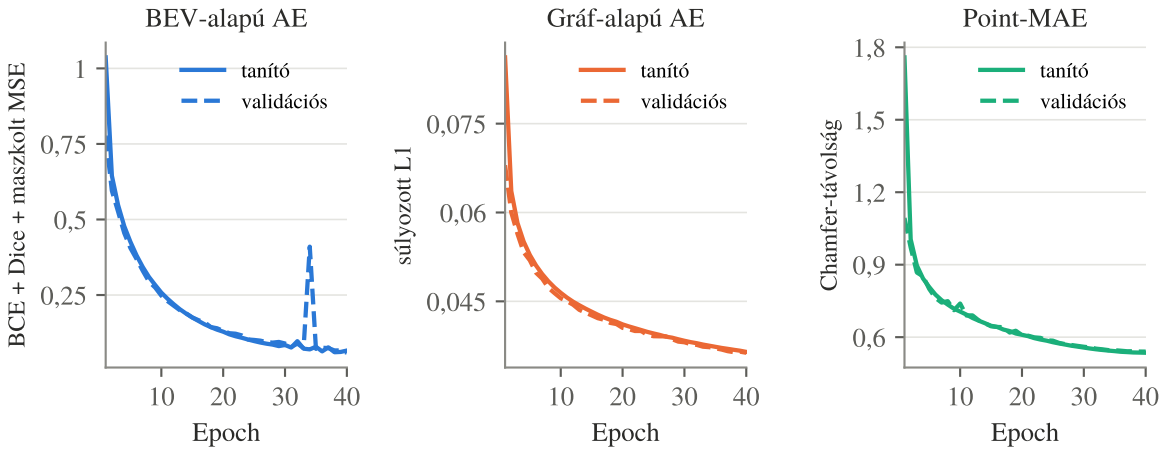
4.3. LiDAR-autoenkóderek

Amint a fentiekben is említettem, az általam választott második gépi érzékelési eljárás a térbeli letapogatás a LiDAR szenzor által. Az ágens LiDAR-ja a CARLA beépített, sugárkövetéses `sensor.lidar.ray_cast` szenzora. Helyzetét tekintva jármű tetején, középpontja felett 2,4 m magasan helyezkedik el. A szenzor 64 csatornás, függőlegesen $+10°$ és $-25°$ között, vízszintesen 360°-ban, 50 m-es hatótávolságig mér. Forgási frekvenciáját nézve a szimulációval azonos 20 Hz-re állítottam, így minden szimulációs lépésben pontosan egy teljes körülfordulásra elegendő adata érkezik. Egy fordulat 64×1024 sugárból áll, ezek egy részét a CARLA a valós szenzorokhoz hasonlóan véletlenszerűen elejti, így képkockánként mintegy 30 ezer pont marad.

A 3D pontfelhő általánosságban rendezetlen ponthalmazként érkezik a szenzorból. A pontok száma jelenetről jelenetre változik, ez a változatosság egy neurális háló számára közvetlenül nem feldolgozható. A kameránál látottakhoz hasonlóan itt is az adat látens térbeli beágyazása volt a tervem. Szintén választottam három különböző architektúrájú autoenkódert majd ezeket mértem össze, a lényegi különbség, hogy mindegyik más formában kapja meg a jelenetet: felülnézeti rácsként, távolságképként, illetve közvetlenül pontfelhőként (10. ábra). Mindhárom modellt azonos adathalmazon, 40 epochon át, Adam optimalizálóval tanítottam, és a validációs halmazon legjobbnak bizonyult modellt mentettem el. A hibafüggvény modellenként eltér, mert mindegyik más formában állítja vissza az adatokat. A hiba alakulását a 11. ábra mutatja, a validációs hiba egyik modellnél sem kezdett tartósan nőni.



10. ábra. Ugyanaz a LiDAR-képkocka a három modell bemeneti formájában; a háromszög a saját járművet jelöli, a távolságképen a fehér cellákból nem érkezett visszaverődés

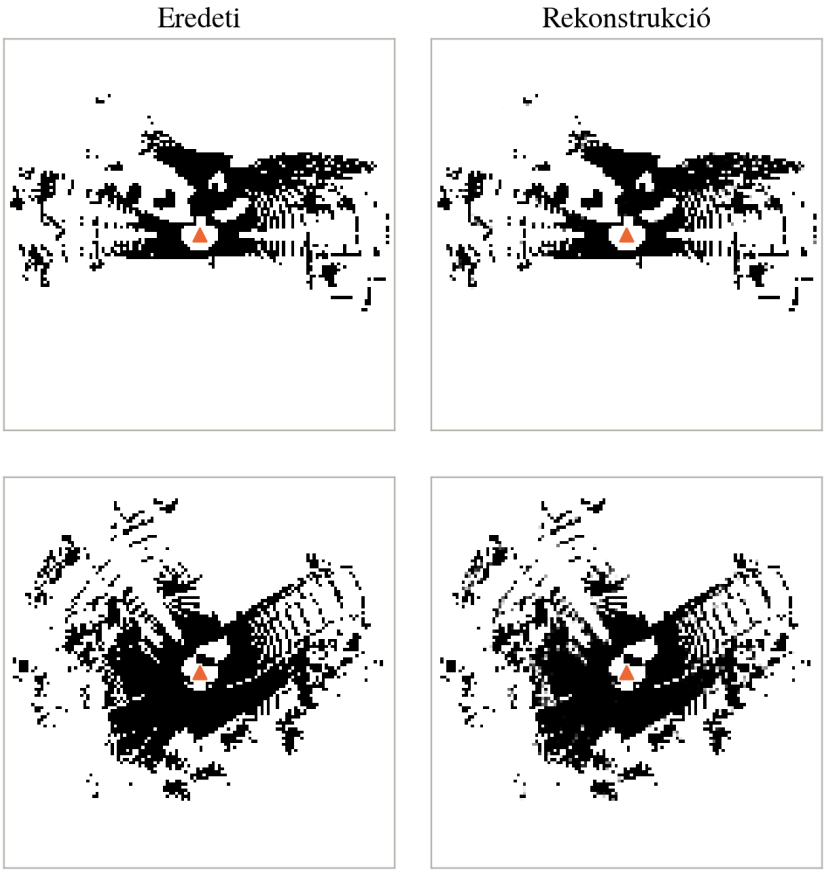


11. ábra. A LiDAR-autoenkóderek tanító és validációs hibája epochonként; az eltérő hibafüggvények miatt modellenként külön tengelyen

4.3.1. BEV-alapú autoenkóder

Első megoldásként a pontfelhőt felülnézeti (*bird's-eye view*, BEV) rácsra vetítését használtam, ami a LiDAR-adatok feldolgozásában gyakran alkalmazott eljárás. A jármű körüli 102,4×102,4 m-es területet 0,8 m-es cellákra osztottam, így 128×128 cella keletkezett, majd cellánként három értéket számoltam.Elsőként foglalt-e, majd megnéztem benne lévő legmagasabb és legalacsonyabb pont magasságát. Erre a már kétdimenziós képre egy konvolúciós autoenkódert építettem, amely a képet egy jóval kisebb jellemzőtérképbe tömörítette. A foglaltság 0/1 érték, ezért erre bináris keresztentrópiát és Dice-veszteséget, a két magassági csatornára pedig csak a foglalt cellákon számolt, maszkolt MSE-t használtam. A kötegméret 768, a tanulási ráta 10^{-3} volt.

A rekonstrukció (12. ábra) szinte hibátlan, a foglalt cellák 98–99%-át helyesen adja vissza. Sajnos ennek a megoldásnak több hátránya is, hogy a rács felbontása rögzített, a 0,8 m-nél kisebb részletek elvesznek, valamint a függőleges 3D szerkezetek nagy része már a bemenetből hiányzik. A kódoló kimenete, vagyis a látens reprezentáció egy 256×8×8-as jellemzőtérkép, ami összesen 16 384 érték.



12. ábra. A BEV-alapú autoenkóder rekonstrukciói; a foglalt cellák feketék

4.3.2. Gráf-alapú autoenkóder

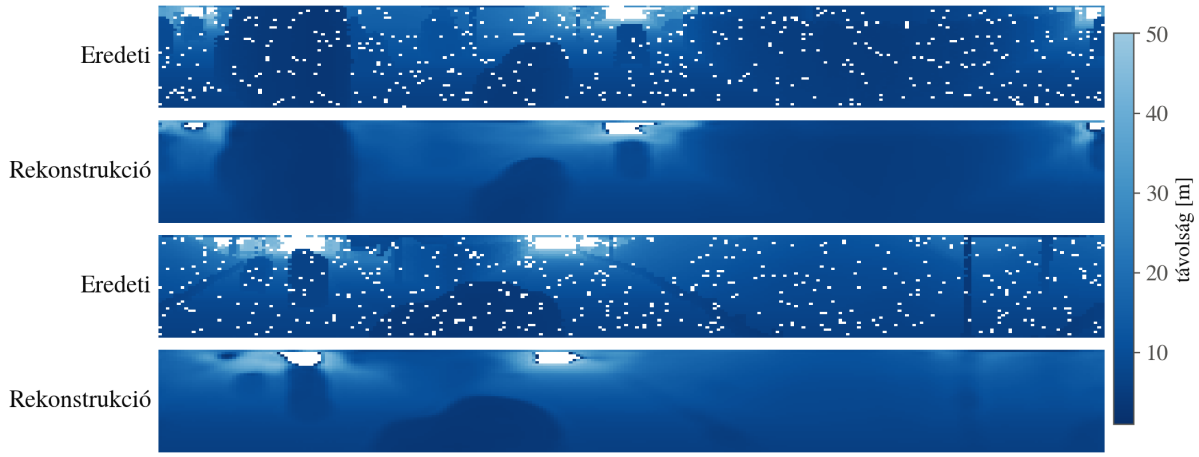
A második megoldás során a pontfelhőt távolságképpé alakítom. Ez annyit tesz, hogy minden sugárirány egy cella, az oszlop lesz a hozzá tartozó irányszög, a sor pedig az emelkedési szög. A cellák értékét pedig a mért távolság adja meg. A CARLA LiDAR-ja szabályos sugárrácsban mér, ezáltal maga a vetítés kevésbé veszteséges, mint a BEV esetén. Az ég felé tekintő felső sorok nem rendelkeznek számomra fontos információval, ezért levágtam. Az így keletkezett kép 44×256 cellás, a távolságokat pedig logaritmikus skálán a $[-1,1]$ tartományba normáltam.

A kódoló ötletét a TopoLiDM [22] LiDAR-modellből vettem. A hálózat a konvolúciós rétegek mellett gráfrétegeket is tartalmaz, ezek a rétegek minden jellemzőt a hozzá leginkább hasonló szomszédjaival vetnek össze, így a térben összetartozó részeket akkor is együtt kezelik, ha a képen távol esnek egymástól. A kép két szélét körkörös konvolúció köti össze, hiszen a 360°-os körben való szomszédság fontos részlet.

A kódoló kimenete $16\times11\times32$-es jellemzőtérkép. Hibafüggvénynek az L1-et választottam, amely átlagos abszolút hibát számolja:

$$L_1 = \frac{1}{N}\sum_{i=1}^{N} |x_i - \hat{x}_i|\,, \tag{4}$$

A képletben x_i az eredeti, $\hat{x}_i$ a visszaállított távolság az i-edik cellában. Az N pedig a cellák száma, az eljárás minden cella eltérését az eltérés nagyságával arányosan veszi figyelembe, így egy-egy nagyobb hiba nem nyomja el a többit, valamint a ritka, kiugró mérési hibák sem befolyásolhatják a tanulást. A hiba akkor a legkisebb, amikor a modell cellánként a legvalószínűbb értéket, vagyis a mediánt becsli. Ez a távolságképen különösen fontos, mert egy tárgy széle éles ugrásként jelenik meg. Az üres, visszaverődés nélküli cellák kisebb, 0,25-ös súllyal számítanak, mert a kép jelentős részét kitevő üres területek különben elnyomnák a valódi távolságok hibáját. Mivel így is beleszámítanak, a modell azt is megtanulja, hol végződik egy tárgy. A kötegméret 32, a tanulási ráta $4{,}5\cdot10^{-6}$ volt. A rekonstrukción (13. ábra) a közeli járművek és az út széle jól kivehetők, a finom részletek viszont láthatóan kevésbé maradnak hangsúlyosak.



13. ábra. A gráf-alapú autoenkóder rekonstrukciói két képkockán; a sötétebb szín közelebbi pontot jelöl, a fehér cellából nem érkezett visszaverődés

4.3.3. Point-MAE

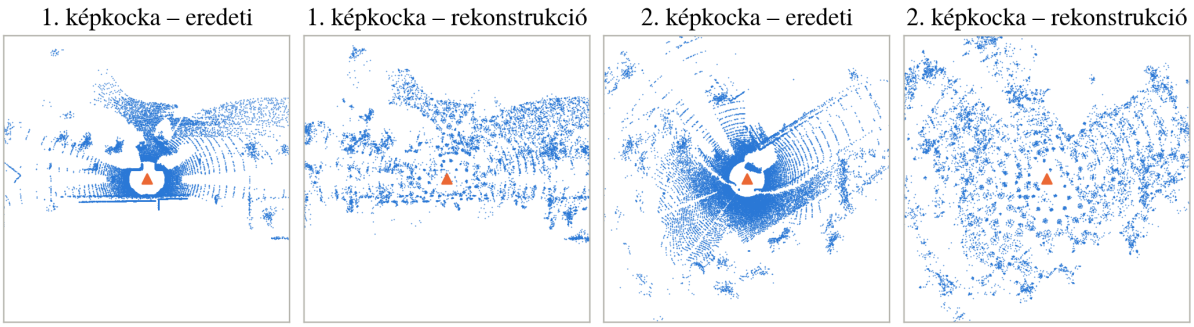
A harmadik választottam a Point-MAE [23] modell ami képeknél már sikeresen bevált maszkolt autoenkóder ötletét viszi át pontfelhőkre, és közvetlenül a pontfelhőn dolgozik, képkockánként 24 576 véletlenszerűen kiválasztott ponttal.

A pontfelhőt 512 darab, egyenként 32 pontos foltra bontja. Ezek után a foltok középpontjait legtávolabbi-pont mintavételezéssel választja ki. Erre azért van okvetlenül szükség, hogy egyenletesen lefedjék a jelenetet. Minden ponthoz a 32 legközelebbi szomszédot rendeli. Minden foltot egy kis háló egy 192 elemű vektorrá, úgynevezett tokenné alakít, a foltok helyét pedig külön pozíciókódolás segítségével adja meg.

Ezen tokeneket egy Transformer kódoló dolgozza fel. Az eljárás során a foltok 60%-át elrejti, és a modellnek a látható foltokból kell visszaállítania a rejtetteket. Ezen modell esetén a hibát Chamfer-távolsággal mértem, mely mindkét pontfelhő minden pontjához megkeresi a másik felhő legközelebbi pontját. Ezeknek a pontoknak a távolságát átlagolja, ezáltal nem függ a pontok sorrendjétől. A kötegméret 40 volt, a tanulási ráta rövid felfutás után koszinuszos ütemezéssel $2 \cdot 10^{-3}$-ról $2 \cdot 10^{-5}$-re csökkent. A kódoló kimenete, vagyis a látens reprezentáció 512 token, egyenként 192 dimenzióval, ami összesen 98 304 érték.

Ez jelentősen több, mint maga a bemenet, amely 24 576 pont $\times$ 3 koordináta, azaz 73 728 érték, vagyis ez a modell nem tömörít. A maszkolt autoenkóder nem szűk keresztmetszettel, hanem a kitakart foltok visszajóslásával tanult így mint kiderült számomra használhatatlan lett. A legtávolabbi-pont mintavételezés ráadásul véletlen pontból indul, ezért ugyanaz a képkocka kétszer kódolva is más sorrendű, részben más foltokból álló tokensort ad.

A rekonstrukción (14. ábra) a jelenet nagy vonalai felismerhetők, de a pontok csomókba tömörülnek, és a járművek körvonala elvész.



14. ábra. A Point-MAE rekonstrukciói felülnézetből; a háromszög a saját járművet jelöli

4.3.4. Összehasonlítás és a választott modell

A megfelelő LiDAR autoenkóder keresése elején három modellt készítettem, de a Point-MAE-t kihagytam az összehasonlításból, mert mint kiderült nem lett megfelelő ahhoz, amire én használnám. A 98 304 értékből álló látense nagyobb, mint maga a bemenet, vagyis nem tömörít.

Így a következőkben a másik két modellre fókuszálnék. Mindkét megoldás más-más bemenetet állít vissza, ezért a rekonstrukciós hibájuk nem vethető össze.

Ezen okból másképp fogtam meg azon kérdést, hogy vajon hogyan is válasszam ki a legjobb rekonstrukciót. Az alapgondolat onnan indult, hogy az ágens ugye nem a rekonstrukciót kapja meg, hanem a kódoló kimenetét, a látens minőségének elemzésére fektettem a hangsúlyt. Legfőbb célom volt kideríteni, hogy melyik eljárás képes a legjobban megőrizni a környezet geometriáját. Összesen három mérést végeztem az adathalmazból véletlenszerűen kiválasztott 2000 képkockán.

Az eljárások a következők:

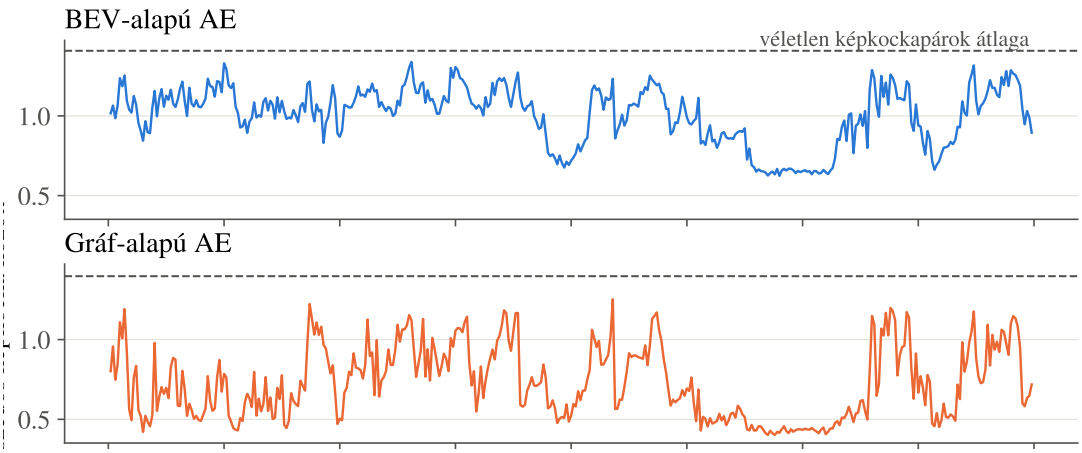
- **Effektív rang** (RankMe [24]): a látens dimenziók mekkora része hordoz, gyakorlatilag azt mérem mennyire zajos a látens vektor; az 1-es érték teljes kihasználtságot jelent.
- **k-NN konzisztencia** [25]: a látens térben legközelebbi képkockák valóban hasonló jelenetet mutatnak-e; a szomszédok pontfelhői hányszor közelebb vannak egymáshoz, mint két véletlen képkockáé.
- **Időbeli simaság**: két egymást követő képkocka látense hányszor közelebb van egymáshoz, mint két véletlen képkockáé. Ha alig, a látens apró változásra is nagyot ugrik, ami az ágens számára zajos bemenet.

A gráf-alapú modell látense a legsimább (15. ábra): két egymást követő képkocka között nála a legkisebb a változás. Az effektív rangban a BEV-alapú modell a legjobb, de mindkét modell 0,5 felett van, tehát egyik látense sem omlott össze néhány dimenzióra.

A fenti mérésekből következtetve a gráf-alapú autoenkódert választottam. Ennek a látense követi a legpontosabban a jármű folyamatos mozgását. Ez a tulajdonság kiemelkedően fontos az

2. táblázat. A két LiDAR-autoenkóder látens reprezentációjának összehasonlítása

Modell	RankMe	k-NN	Simaság
BEV-alapú AE	**0,725**	**1,51**×	1,44×
Gráf-alapú AE	0,504	**1,51**×	**1,86**×



15. ábra. A látens változása két szomszédos képkocka között, 400 egymást követő, 5 Hz-cel rögzített képkockán; a szaggatott vonal két véletlen képkocka látensének átlagos távolsága

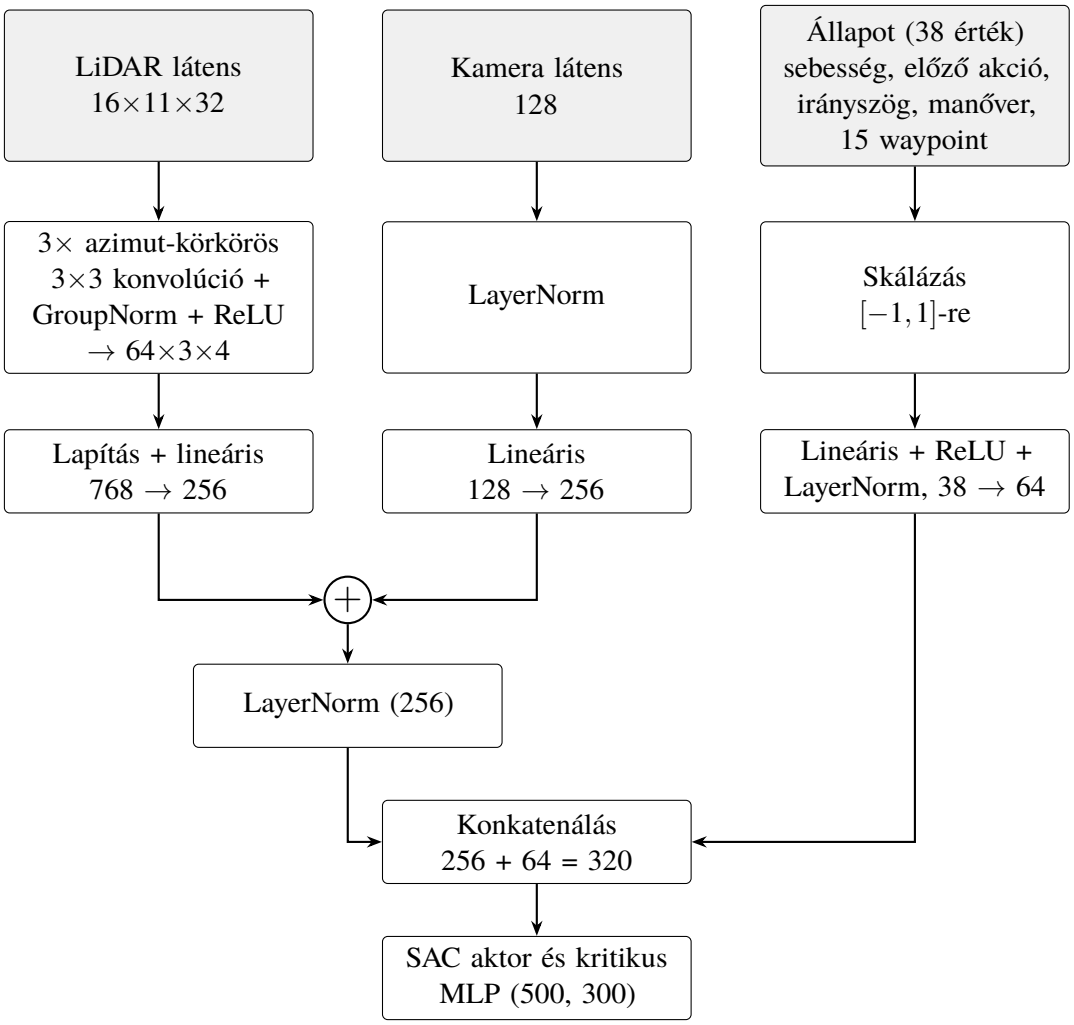
ágens számára, mivel az egymást követő lépésekben következetes bemenetet kap.

4.4. Szenzorfúzió

A két autoenkóder látense jóval nagyobb, számokban is és nagyságrendileg is, mint az ágens többi bemenete. A LiDAR látens 16×11×32, azaz 5 632 érték, a kamera látense 128 érték, míg a jármű állapotát leíró megfigyelések összesen 38 számszerű értéket tartalmaznak.

Amikor ezek összefűzését az SB3 beépített jellemzőkinyerőjére bíztam, a nagy méretű látensek elnyomták a többi bemenetet, és az ágens nem tanult meg érdemben vezetni. Ezért a LiDAR-látenst külön ágon tömöríteni kellett, mielőtt a többi bemenettel egyesítettem.

Ezen problémától motiválva saját jellemzőkinyerőt (*feature extractor*) készítettem, amely három külön bemeneti ágon dolgozza fel a szenzorokból érkező adatokat, majd ezekt a végén ugy rendezi ahogy nekem tetszik (16. ábra).



16. ábra. A saját jellemzőkinyerő felépítése; a szürke dobozok a bemenetek

LiDAR ág. A LiDAR szenzor kódolójából származó látens nem egy sima vektor, hanem egy többdimenziós jellemzőtérkép. Ezt az ágens saját konvolúciós rétegei dolgozzák fel, amelyek az ő tanulása folyamán finomhangolodnak azon jellemzőkre amire tényleg szűkségesek a modellnek gyakorlatilag együtt tanul a modellel. Három 3×3-as konvolúciós réteg csökkenti a térkép méretét 16×11×32-ről 64×3×4-re, mindegyik után GroupNorm normalizálással és ReLU aktivációval. A kapott 768 értéket egy lineáris réteg 256 dimenzióra vetíti.

Kamera ág. A kamera látense már vektor, ezért itt elég egyetlen lineáris réteg, de a fentiekben is említettem, hogy a LiDAR jellemzők 256 dimenziósra csökkentek így ha össze akarom adni a 128 hosszú látenssel akkor 256 dimenziósra fel kell skáláznom. Ezt szintén egy lineáris réteggel oldtam meg.

Fúzió. A lényeges rész a fuzios eljárás maga esetemben a két szenzorágat összeadással vezettem össze. Azért ezt a megoldást preferáltam az összefűzéssel ellentétben, mert a jellemzőkinyerő végén a kimenet alapértelmezetten konkatenálással áll össze, fölösleges még egy összefűzést itt is alkalmazni. Mivel a két ág azonos, 256 hosszú vektort ad, az összeadáshoz nincs szükség további vetítő rétegre, és az egyesített szenzorjellemző-vektor mérete sem nő. Az összeg után egy LayerNorm rögzíti a vektor teljes skáláját, mert a gráf-alapú kódoló kimenete sem korlátos. Viszont a két szenzorág egymáshoz viszonyított súlyát nem állítom manuálisan, ezt az ágensre bízom, honnan tud több információra szert tenni.

Állapot ág. A maradék további megfigyelés került utoljára A jármű sebessége, az előző lépés kormány- és gázértéke, a következő waypoint irányszöge, a manőver one-hot kódolva, valamint a következő 15 waypoint relatív koordinátái.

Ezeket a jellemzőket a megfigyelési tér határai alapján a $[-1, 1]$ tartományba skáláztam, hogy a nagy értékű jelek ne nyomják el a kicsiket, majd egy lineáris réteg, ReLU és LayerNorm 64 dimenzióra képezi le őket.

A jellemzőkinyerő kimenete a 256 hosszú szenzorvektor és a 64 hosszú állapotvektor összefűzése, összesen 320 érték. Ez kerül a SAC aktor- és kritikushálózatának bemenetére, amelyek két, 500 és 300 neuronos rejtett rétegből állnak. Így a szenzorinformáció a bemenet nagyobb részét adja, de az állapotjelek is külön, saját ágon jutnak el a döntéshozó hálózatig.

5. Jutalomfüggvény

Kutatásom fő kérdése, hogy milyen jutalomfüggvénnyel tanítható meg az ágensnek a biztonságos sávtartás, valamint a nála lassabb járművek szabályos, balról történő megelőzése, ha erre lehetőség van. Ennek vizsgálatára öt különböző jutalomfüggvényt készítettem és hasonlítottam össze. Ezeket R1-től R5-ig jelölöm, és a velük tanított modelleket is így nevezem. Az R1–R3 függvények részletesebben minden forgalmi helyzetre külön, kézzel általam formált tagot tartalmaznak, ahol minden tagnak saját célja van. Az utolsó két esetben, az R4 és az R5 függvénynél másféle megközelítést alkalmaztam, amely elhagyja a kézi tagokat, egyetlen fizikai mennyiséget jutalmaz.

Ugyanakkor a különböző ágensek tanítása során voltak közös konstans metrikák mint például, az epizódzáró büntetés. A járművel való ütközés esetén -30, a többi esetben, vagyis az útról való letérésnél, a megállásnál és a sebességhatár túllépésénél -10. Már az is büntetés az ágensnek, hogy véget ér az epizód, mert onnantól nem kap több jutalmat, a -10 pedig ehhez további büntetést ad. Az ütközés a legsúlyosabb hiba, ezért annak háromszoros a büntetése, így az előzésért járó $+5$ miatt sem éri meg kockáztatni. Minden modell esetén közös a kormányrángatásért kiszabott büntetés, amelyet a kormányjel lépésenkénti változásának kétszereséből számolok. A kettes szorzót úgy választottam, hogy egy hirtelen 0,1-es kormánymozdulat 0,2 levonással járjon, ez a szabad haladásért kapott lépésjutalom ötöde. A kis korrekciókért viszont alig von le valamit. Az

előzési manővert felismerő állapotgép, ahol csak következő van engedélyezve a balra, szaggatott vonalon áthaladás ha szabad a sáv kihúzódhat és megkísérelheti az előzési manővert.

5.1. R1: sávtartás és szabályos előzés

Az alapjutalom amit kap szabad úton négy, nullától egyig terjedő tényező szorzata:

$$r = v^{1,9} \cdot c \cdot a \cdot s, \tag{5}$$

ahol v a sebességtényező, amely 50 km/h-nál 1, alatta lineárisan csökken. A c a sávközép-tényező, amely a sáv közepén 1, attól 2 m-re 0, közte lineáris. Az a az irányszög-tényező, amely az útvonal irányába nézve 1, arra merőlegesen 0, közte szintén lineáris. Az s pedig a simasági tényező, amely azért felelős, hogy a modell ne végezzen hirtelen fölösleges kormánymozdulatokat, amikor a sávjában közlekedik. Az s-t úgy számolom, hogy veszem a sávközéptől mért távolság szórását az utolsó 30 lépésben, vagyis 1,5 s alatt: ha a szórás 0, akkor $s = 1$, 0,35 m-nél pedig már 0. A 2 m azért jó határ, mert ez kicsit több a körülbelül 1,75 m-es fél sávszélességnél, tehát ott már a szomszéd sávban van az autó. 0,35 m-es szórásnál pedig már szemmel is látszik, hogy cikázik a sávon belül. A tagokat szorzás utolján fuzionálom ennek oka, hogy bármelyik kiesése nullázza a jutalmat. Az ágenst így kényszerítem arra, hogy egyszerre kell mindegyiket teljesítenie. Az 1,9-es kitevő oka a következő, az 1-nél nagyobb kitevő hatása az, hogy a lassú haladás aránytalanul keveset ér: 25 km/h sebességnél $v = 0{,}5$, a sebességtag értéke viszont már csak $0{,}5^{1,9} \approx 0{,}27$, nem pedig a fele. Ez erősebben ösztönöz a sebesség kihasználására, mint egy lineáris tag tenné, ugyanakkor az előzéskor is hatékonyabb mert vonzobbá teszi mert végsősoron a cél, hogy az ágens elkerülje a lassan forgalmirésztvevőket.

A következő eset, amikor az ágens előtt van egy forgalmi szereplő, de nem engedélyezett az előzés. Ha a saját sávjában 40 m-en belül lassabb jármű megy előtte, a szomszédos sáv pedig foglalt vagy záróvonal választja el, akkor követnie kell nem szabad baleset történjen. Ilyenkor a sebesség helyett a biztonságos követési távolság megtalálását jutalmazom, hogy az ágens megfelelően kövesse azt, aki előtte van. A 40 m 50 km/h-nál nagyjából 3 s, ez több, mint a 2 s-os, körülbelül 28 m-es követési távolság, így az ágensnek van ideje eldönteni, hogy követ vagy előz.

$$r_{\text{követés}} = 0{,}8\, g(h)\, m \cdot c \cdot a \cdot s + p_{\text{TTC}}, \tag{6}$$

ahol h a követési időköz, vagyis az előtte haladótól mért távolság és a saját sebesség hányadosa, g pedig egy lognormális alakú függvény, amelynek maximuma 2 s-nál van. A 0,8-as szorzót azért alkalmazom, hogy a tökéletes követés kevesebbet érjen a szabad haladásnál. A sáv közepén, 2 s-os időközzel, az előtte haladóval azonos sebességgel követve a lépésjutalom 0,8, míg szabad úton 50 km/h-val 1,0. Így az akkor is megéri előzni ha egyébként hibátlanul követ, ezzel kiküszöbölve, hogy szabad út esetén merjen előzni az ágens.

Az m az ágens és az előtte lévő jármű sebességének hányadosa, amely felülről 1-re van korlátozva:

$$m = \min\left(\frac{v_{\text{ego}} + 1}{v_{\text{lead}} + 1},\ 1\right). \tag{7}$$

ez szükséges mivel alapból a g kijátszható. A h követési időköz a távolság és a *saját* sebesség hányadosa, ezért lassítással is növelhető. Ha az ágens lefékezne és lemaradna, h 2 s fölé nőne, a jutalma mégsem csökkenne. Az m ezt hivatott javítani a lassabb haladás arányosan bünteti, az előtte haladó sebességét elérve, hogy kövesse amíg kell utána már ne érje meg követni. A két tényező együtt azt várja el, hogy az ágens tartsa a kétmásodperces követési időt, és közben haladjon ugyanolyan gyorsan, mint az előtte lévő jármű. A számlálóban és a nevezőben szereplő +1-es eltolás technikai okból kell, nélküle álló jármű esetén nullával kellene osztani.

A kétmásodperces optimum és a g lognormális alakját a szakirodalomból vettem. A h időköz csak pozitív lehet, eloszlása pedig jobbra ferde, mivel az autóvezetők nagyrészt inkább közelebb haladnak az előttük haladóhoz, mint lemaradjanak. Az empirikus forgalmi mérések kimutatják ezt amit a lognormális eloszlás írja le a legjobban [26], a g-ben szereplő $\sigma = 0{,}4365$ szórás pedig az NGSIM adathalmazon, 1341 valós követési eseményre illesztett eloszlás becsült paramétere [27]. A jutalom így a tényleges emberi vezetői viselkedéshez igazodik.

A p_{TTC} (*time to collision*, TTC) büntetés azt a helyzetet kezeli, amit a g nem lát. A pillanatnyi követési időköz lehet ideális úgy is, hogy az ego jármű közben gyorsan közeledik az előtte haladóhoz, tehát ütközés veszélye áll fenn. A büntetés az ütközésig hátralévő idő alapján jár, 7 s alatt lép életbe $0{,}3 \cdot \ln(t/7)$ alakban, és 1,5 s-nál éri el a $-0{,}46$-os maximumát. Ha az ágens nem közeledik az előtte haladóhoz, ez az idő végtelen, és büntetés sem jár. Ha az ágens 50 km/h-val megy egy 25 km/h-s autó mögött, a 7 s kb. 48 m, tehát a büntetés már akkor elindul, amikor 40 m-ről meglátja az előtte lévőt. A logaritmus miatt minél kisebb a TTC, annál gyorsabban nő a büntetés, ezért 1,5 s-nál levágom, különben nulla közelében végtelen lenne. A 0,3-as súlyt úgy állítottam be, hogy a legnagyobb, $-0{,}46$-os büntetés kb. a fele legyen a jó követésért járó 0,8-nak, így a követési jutalom marad a fontosabb.

Előzési manőver végrehajtása közben adott lépésenkénti jutalom

$$r_{\text{előzés}} = (0{,}3 + 0{,}6\, v^{1{,}9}) \cdot l \cdot k, \tag{8}$$

ahol l a célsáv közepétől mért távolságot bünteti, k pedig az elhúzódó manővert: 15 s után, illetve az elhaladást követően sávonként 3 s után négy másodperc alatt 1-ről 0,3-ra esik. A 15 s-ot szándékosan hagytam bőven: 25 km/h-s sebességkülönbségnél egy előzés kihúzódással és visszasorolással együtt nagyjából 8 s, így egy rendes előzést ez még nem büntet. Az elhaladás után sávonként 3 s elég arra, hogy nyugodtan visszasoroljon, a 4 s alatti fokozatos csökkenés pedig azért van, hogy a jutalom ne essen le egyszerre. A k azért nem megy 0,3 alá, mert néha a forgalom miatt tényleg nem tud rögtön visszamenni, ilyenkor nem akarom büntetni, csak kevesebbet kap,

mint a saját sávjában. A képletben szereplő 0,3-as alaptag biztosítja, hogy a lassú előzés is jobban megérje, mint a lassabb jármű mögötti beragadás. A szabályosan befejezett előzésért járművenként egyszer +5 bónusz jár. Ez jóval kevesebb a −30-as ütközési büntetésnél, így egy kockázatos előzés miatt nem éri meg. Azért jár csak egyszer járművenként, hogy az ágens ne tudja ki-be sorolgatással gyűjtögetni.

Külön tag bünteti azt, ha az ágens előzhetne, de nem teszi: ha 25 m-en belül halad az előtte lévő mögött, és nem gyorsabb nála legalább 5 km/h-val, lépésenként

$$r_{\text{beragadás}} = -0{,}1 \cdot \big(1 + 0{,}4\,(1 - v) + 0{,}4 \cdot \min(t_{\text{b}}/3,\ 1)\big) \tag{9}$$

büntetést kap, ahol t_{b} az egyhuzamban beragadva töltött idő másodpercben. A büntetés így teljes sebességnél, a beragadás elején −0,10, álló helyzetben, legalább 3 s beragadás után pedig −0,18, tehát minél lassabb az ágens és minél tovább ragad be, annál nagyobb. Az együtthatókat kézzel hangoltam be. A −0,1 kicsi a szabad haladásért járó legfeljebb 1,0-hoz képest, így nem a büntetés irányítja az ágenst, csak nem hagyja, hogy beragadjon a lassabb autó mögé. A lassúság és a beragadás ideje külön-külön legfeljebb 40%-kal növeli a büntetést. Az időtag 3 s alatt éri el a maximumát, így egy rövid lassítás alig számít, a hosszú beragadás viszont már igen. Az 5 km/h-s határral azt választom szét, hogy az ágens tényleg felzárkózik az előtte lévőre, vagy csak beállt mögé, 25 m-en belül pedig már nem közeledik hozzá, hanem mögötte megy. Ez az egyetlen eset, amikor a lépésjutalom nem szorzódik a sávtartási tényezőkkel: ellenkező esetben a rossz sávtartás csökkentené a büntetés mértékét.

Ha az ágens rossz sávba megy, vagyis jobbra húzódik ki, vagy előzés nélkül átmegy a bal sávba, lépésenként −0,5 büntetést kap. Ez sokkal több, mint a beragadás legnagyobb, −0,18-os büntetése, tehát inkább maradjon a lassabb jármű mögött, mint hogy szabálytalanul előzzön. A jelenetet viszont nem zárom le, hogy meg tudja tanulni, hogyan jöjjön vissza a saját sávjába.

5.2. R2: ütközésidő-büntetés előzhető helyzetben

Következőkben itt a második függvény hangolásom, R1-ben a p_{TTC} büntetés csak akkor járt, ha az előzés nem volt lehetséges. A TTC jutalom definícióját megpróbáltam átültetni, hogy előzési manőver során is alkalmazható legyen. Az R2 egyetlen változtatása, hogy ugyanezt a büntetést ugyanazzal a 7 s-os küszöbbel és 0,3-es súllyal az előzhető állapotra is kiterjesztem. A büntetés a kihúzódás első méterén fokozatosan szűnik meg, így ha már megkezdi a manővert, jutalmazom.

5.3. R3: hangolás bátrabb előzésre

Harmadik jutalomfüggvényem azt hivatott javítani, ami az előzőben problémává vált. Az ágens nem mert előzni vagy csak nagyon későn, az R2-vel tanított modell gyakorlatilag nem mert előzni.

Ahogy a 3. táblázat megmutatja, a lassabb jármű mögött 25–40 m-rel lemaradva haladni jobban megérte az ágensnek büntetlen volt, sőt pozitív jutalmat adott.

3. táblázat. Lépésjutalmak egy 25 km/h sebességgel haladó jármű mögött, szabad bal sáv mellett. Az értékek a sáv közepén, egyenes haladásnál, kormánybüntetés nélkül érvényesek.

Helyzet	R2	R3
Beragadás 25 km/h-val, 25 m-en belül	−0,12	−0,12
Lemaradás 25–40 m-rel, 25 km/h-val	+0,27	−0,12
Ráfutás 50 km/h-val, 30 m-re	0,86	1,00
Ráfutás 50 km/h-val, 15 m-re	0,65	0,80
Előzés 50 km/h-val a bal sáv közepén	0,90	1,10
Előzés 25 km/h-val a bal sáv közepén	0,46	0,59

Miközben az előzés a legjobb esetben is csak 0,90-et ért, kevesebbet, mint a saját sávban elérhető 1,00.

Ezeket elnézve a következő négy ponton változtat az R3:

1. Az előzhető helyzetben járó ütközésidő-büntetés küszöbe 7 s-ról 3 s-ra csökken, súlya viszont 0,3-ról 0,6-ra nő. Így csak a tényleges késlekedést bünteti, a gyors ráfutást nem.

2. Az előzés lépésjutalma $(0{,}3 + 0{,}6\,v^{1{,}9})$ helyett $(0{,}4 + 0{,}7\,v^{1{,}9})$, vagyis 50 km/h-nál 0,90 helyett 1,10. Így próbáltam a kihúzódást előzés helyzetre jobban jutalmazni.

3. A lemaradás büntetése nemcsak 25 m-en belül jár, hanem 40 m-en már ez a teljes érzékelési tartomány.

4. A +5 bónuszt két részre bontottam így megkapja de máson lesz a hangsúly. A +2-t az elhaladáskor és +3 a szabályos visszasoroláskor. A 0,98-as diszkonttényező mellett az eredeti, csak a manőver végén érkező bónusz a kihúzódás pillanatában mindössze 0,66-ot ért; a kettébontás ezt körülbelül 1,13-ra növeli.

5.4. R4: haladásalapú jutalom

Negyedik függvény esetén úgy döntöttem, teljesen átszabom a modell jutalmazási logikáját. Az R1–R3 függvények minden forgalmi helyzet esetén, kézzel hangolt tagot tartalmaztak. Az R4 ezekkel ellentétben egyetlen fizikai mennyiséget jutalmaz. Ez a következő az útvonal mentén megtett elmozdulást. A lépésjutalom minden megengedett sávban

$$r = v \cdot \cos\vartheta \cdot l \cdot k, \tag{10}$$

ahol v a sebességtényező, ϑ az útvonal irányától való eltérés szöge, l és k pedig a (8) egyenletből ismert sávtartási és elhúzódás-tényező. A koszinuszos tag a ferde haladást veszi figyelembe, visszafelé haladva negatív. A sebességben lineáris, tehát 25 km/h-nál feleannyi jutalom jár, mint 50-nél.

A feltételezésem szerint így a lassabb jármű mögött a haladás magától kisebb, ezért az előzés kézi formálás nélkül is megéri. Ezek által a kézzel formált jutalmakat kivettem az egyenletből , ami megmaradt az a feladat szabálya nem pedig formálás. Az epizódzáró büntetések, az előzési állapotgép, a rossz sávért járó $-0{,}5$, a kormányrángatás büntetése és a szabályos előzésért járó $+5$ bónusz. Az ütközést így kizárólag a -30 terminális büntetés tanítja.

Ebben a függvényben a bal sáv önmagában nem ér többet a sajátnál. Például egy 25 km/h-s autó mögött lépésenként 0,5 jár, előzés közben 50 km/h-val 1,0, az ágens azért előz, mivel ez által gyorsabban halad. Külön csak a $+5$ jutalmazza az előzést, ez viszont csak a célt jelöli ki, a módját nem. Meglátásom szerint az eljárás jobb lehet mivel, nincsenek a kézi tagok esetében előfordulhat, hogy a jutalmak bizonyos esetekben kioltják egymást.

5.5. R5: potenciálalapú jutalomformálás

Az ötödik függvény az előzőre épül, csak kiegészíti egy darab plusz taggal Az ötletet Ng, Harada és Russell 1999-es cikkéből vettem [28]. Ez a cikk azt vizsgálja, hogy lehet az ágensnek plusz jutalmat adni úgy, hogy közben ne tanuljon rossz viselkedést. Ez alatt értem, hogy a kézi segítő tagok amik az előző 3 függvényben vannak felmerülhet a probléma, hogy az ágens megkerüli a jutalomszerzés menetét. Az R2-ben például a 25–40 m-es lemaradás még jutalmat is ért, így előzés helyett megérte lemaradni, tehát nem tudott hatékonyan előzni.

Azt bizonyítják, hogy ha a plusz jutalom

$$F(s,s') = \gamma\Phi(s') - \Phi(s) \tag{11}$$

alakú, ahol s az aktuális, s' a következő állapot, γ a diszkonttényező, a Φ potenciál pedig egy szám, amit minden állapothoz hozzárendelek, és azt méri, mennyire jó helyzetben van az ágens, nálam azt, hogy hol tart az előzésben, akkor az ágens ugyanazt a viselkedést tanulja meg a legjobbnak, mint e tag nélkül. A Φ úgy viselkedik, mint egy szint: ha nő, az ágens jutalmat kap, ha csökken, ugyanannyit elveszít. Ha visszaér oda, ahonnan indult, összesen nem nyert semmit. Így ezen tag nem viheti rossz irányba a tanulást, csak annyit csinál, hogy a jutalom hamarabb megérkezik és csak akkor amikor a helyzet úgy hozza.

A módszer implementálására szükségem volt, mert az előző függvényben a $+5$ csak az előzés végén jár, nagyjából 5 s-mal a kihúzódás után. A 0,98-as diszkonttényező mellett ez a kihúzódás pillanatában már csak kb. 0,66-ot ér, vagyis az ágens alig érzi a kihúzódást így nem merte annyira alkalmazni sem.

A potenciált úgy választottam, hogy előzés közben

$$\Phi = 2 + 3\,\varphi, \qquad \varphi = \mathrm{clip}\left(\frac{d_0 - d}{d_0 + 5}, 0, 1\right), \tag{12}$$

legyen, előzésen kívül pedig nulla. A d_0 a megelőzendő autó távolsága a kihúzódáskor, a d az aktuális távolsága, így a φ azt mutatja, hol tart az elhaladásban: a kihúzódáskor 0, és akkor lesz 1, amikor a megelőzött autó már 5 m-rel mögötte van, ezért szerepel a nevezőben $+5$ m. A potenciál így kihúzódáskor 2, az elhaladás végén 5, ami pont a $+5$ bónusz.

Az R4-hez képest tehát nem a cél változik, hanem az, hogy az ágens mikor kapja meg a jutalmat. Ettől elvárásaim alapján gyorsabban és határozottabban tanul meg előzni.

6. Elkészült modellek összehasonlítása

6.1. A kiértékelés módszertana

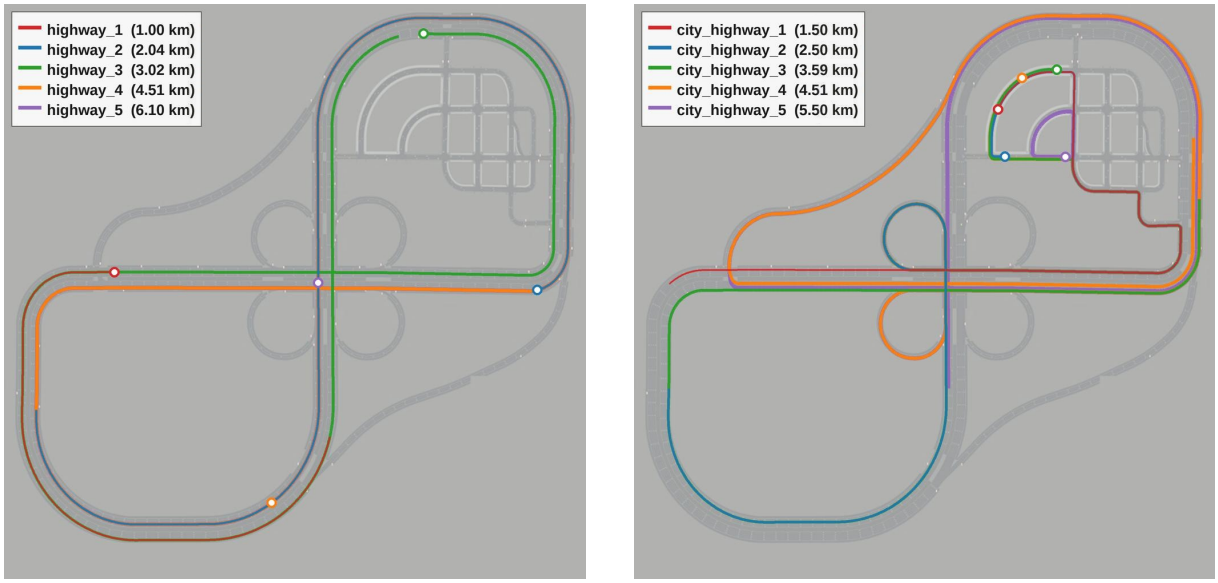
A modellek teljesítményét statisztikai alapon értékeltem, egyetlen futás kimenetele nem jellemzi megbízhatóan a modellt. A modell kiértékelése sztochasztikus, a forgalom résztvevőinek kezdőhelyzete bejárásonként véletlenszerű, így ugyanaz a modell ugyanazon az útvonalon is eltérő helyzetekbe kerül, ez által eltérő eredmények születhetnek. Ezért minden mérést ötször megismételtem ugyanazokon az útszakaszokon.

Tanítás során minden elemet azonosan hagytam, minden esetben a jutalomfüggvényt változtattam. A jellemzőkinyerő, az SAC hiperparaméterek, köztük a $\gamma = 0{,}98$, 20 Hz-es szimuláció, ugyanazok az epizódzáró szabályok, minden modell esetében. Minden esetben a modellek pontosan 1 millió lépésen át tanultak a 100-as kezdőértékkel. Ezekkel a beállításokkal minden esetben kb. 12 óra volt a tanítás. A kiértékelés során a tanítás végén mentett súlyokat mérem össze egymással, determinisztikus akciók sorozatán.

A kiértékelő útszakaszok a beépített térkép tíz előre rögzített szakasza. A szakaszokat próbáltam úgy válogatni, hogy varianciájuk reprezentatív legyen. Öt szakasz autópályát tartalmaz, öt pedig városi környezetből indul, és vezet fel az autópályára (17. ábra). A legrövidebb 1,05, a leghosszabb 6,09 km. Minden modellt ezen a tíz útvonalon ötször értékeltem ki, így összesen 250 bejárás alapján értékeltem a modelleket. A véletlengenerátor ami forgalom résztvevőit szabályozza, a kezdőértékeit magam állítottam be, így adott körben minden modell pontosan ugyanazt a forgalmat kapja. Ez által ugyanazokon a helyzeteken mérődnek össze a modellek.

A kiértékelési folyamat a tanítással ellentétben nem áll le az első hibánál. Ha az autó letér az útról, 5 s-ig áll, vagy 60 km/h fölé gyorsul, felírom a hibát, majd visszahelyezem az útra és megy tovább a jelenet. A megállás akkor nem számít hibának, ha a saját sávjában van előtte jármű, ilyenkor a várakozás a helyes viselkedés.

A járművel való ütközést is felírom, de az nem jár visszahelyezéssel. Ezek adnak egy újabb



(a) autópályás útvonalak (b) városból autópályára vezető útvonalak

17. ábra. A kiértékeléshez használt tíz útvonal a Town04 térképen. A fehér körök az indulási pontokat jelölik, az útvonalak hossza 1,05 és 6,09 km között változik.

mérőszámot, ami kimutatja, hányszor kellett volna beavatkozni. Tíz visszahelyezés után az adott jelenet leáll, ezért a nem egyforma hosszúak, az egyes eseményeket.

A modellek lépéseik során kapott jutalmát nem hasznosítottam az összehasonlításhoz, mert minden modell más jutalomfüggvénnyel tanult. Ezáltal a csak jutalom alapú összehasonlítások nem lennének reprezentatívak. Helyette egyéb módon mérem össze a hasznosságukat, olyan mérőszámokat nézek, amelyek jutalom függvényel nem hoztatok összefüggésbe hanem a modell fizikai viselkedését vizsgálják. Ezek minden modellnél ugyanúgy számolódnak, tehát tényleg vezetési képességeiket tudom összemérni.

Az eredményeket modellenkénti kiértékelését két lépésben átlagoltam. Első lépésben minden útvonalra külön kiszámolok egy értéket az öt bejárásából. Az öt bejárás alatt történt összes eseményt, például az előzéseket vagy az ütközéseket, elosztottam az öt bejárás alatt megtett összes kilométerrel. Ezek által minden útvonalhoz az adott mérőszám egy kilométerre vetített értékét kapom. Második lépésben az útvonalértéknek veszem az egyszerű számtani átlagát.

A módszertanra azért volt szükségem, mivel az útvonalak hossza nagyon eltér, 1,05 és 6,09 km között változik. Ha az összes bejárás eseményeinek egyben venném a számtani átlagát az összes kilométerrel, a hosszú útvonalak sokkal nagyobb súllyal számítanának. A hatkilométeres útvonal közel hatszor annyit számítana, mint egy egykilométeres szakasz. Így a kétlépéses átlagolással minden útvonal ugyanannyit számít, a hosszától függetlenül.

6.2. Eredmények

A modelljeimet az alábbi négy szempont szerint hasonlítottam össze, az adatokat a fent említett és bemutatásra került 10 pályaszakaszon mértem:

- **Sebesség és menetidő:** milyen gyorsan haladt a modell, és mennyi idő alatt ért célba.
- **Előzések:** hány jármű mellett haladt el, ebből hány volt szabályos az előzési állapotgép szerint, és a megkezdett előzések hány százaléka sikerült.
- **Kormányzás és sávtartás:** a kormányjel lépésenkénti átlagos változása, és az idő hány százalékában volt az autó a sáv közepétől 0,5 m-en belül.
- **Ütközések és visszahelyezések:** hány járművet ütött el, és hányszor kellett visszahelyezni az útra, hányszor kellett volna egy embernek beavatkoznia.

A táblázatokban és az ábrákon mind az öt modell egymás mellett szerepel, autópályás és városból autópályára vezető útvonalakra bontva. Adott típusonkénti 25 bejárását összesítve.

6.3. Modellek kiértékelése a szempontok alapján

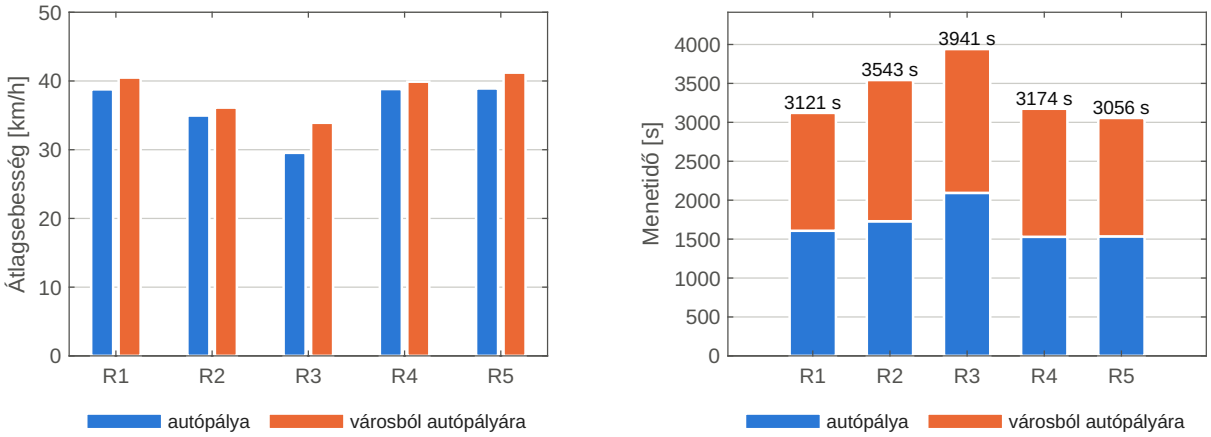
6.3.1. Sebesség és menetidő

4. táblázat. Átlagsebesség és menetidő útvonaltípusonként. A menetidő az útvonalak célba érési idejének összege, útvonalanként a célba ért bejárások átlagával. Az utolsó oszlop a célba ért bejárások száma az 50-ből.

	Átlagsebesség			Menetidő			
	Autópálya	Városból autópályára	Összes	Autópálya	Városból autópályára	Összes	Célba ért bejárás
R1	38,8 km/h	40,5 km/h	39,6 km/h	1607 s	1514 s	3121 s	49/50
R2	35,0 km/h	36,1 km/h	35,5 km/h	1729 s	1814 s	3543 s	50/50
R3	29,5 km/h	33,9 km/h	31,7 km/h	2094 s	1847 s	3941 s	50/50
R4	38,8 km/h	39,9 km/h	39,4 km/h	1530 s	1645 s	3174 s	49/50
R5	38,9 km/h	41,2 km/h	40,1 km/h	1535 s	1521 s	3056 s	50/50

A sebességért felelős 4. táblázat valamint a 18. ábra modellre bontva mutatja be az átlagsebességek közötti különbségeket valamint, hogy mennyi idő alatt tették meg a pályaszakaszokat. A 250 bejárásból összesen 248 célba is ért, a maradék 2 azon esetek amikor többszöri, legfeljebb 10 megengedett próbálkozás után a modell nem tudta befejezni egyáltalán az adott szakaszt.

Kiindulásom az R1 függvény volt, ez szolgált alapul, amely 39,6 km/h-s átlagsebességgel haladt végig a tíz útvonalon. Összesen az 50 bejárását 3121 s alatt teljesítette, valamint 49 alkalommal érte el a célt.



18. ábra. Balra: átlagsebesség útvonaltípusonként. Jobbra: menetidő, a két útvonaltípus összege, a teljes idő az oszlopok felett.

Az R2-ben amely a második modell lett a p_{TTC} büntetést, az R1-ben kizárólag, ha előzni nem lehetett akkor kapott, most az előzhető helyzetre is kiterjesztettem. Eddig itt nem járt TTC-büntetés, az R2-ben viszont ugyanúgy 7 s-os küszöbbel és 0,3-es súllyal kapja, legfeljebb $-0{,}46$-ot 1,5 s-nál. Ennek oka mégpedig az, hogy az ágens előzés során ne közelítse veszélyesen meg az előtte haladót. Ennek hatására érezhetően lelassult, az átlagsebessége 39,6-ról 35,5 km/h-ra csökkent. Értelemszerűen a menetidőn is érezhető lett a kisebb átlagsebesség 3121-ről 3543 s-ra nőtt. Oka, hogy 25 km/h-s sebességkülönbség mellett a 7 s-os küszöb körülbelül 48 m-t jelent, így a büntetés már az előtte haladó jármű észlelésekor életbe lépett, és a modell az előzés helyett inkább lassított és a jármű mögött maradt.

A következőben az R3-ban az előzést próbáltam vonzóbbá tenni. Előzhető helyzetben a TTC-küszöböt 7-ről 3 s-ra csökkentettem, hogy a büntetés csak valóban közeli helyzetben hasson, a súlyát viszont 0,3-ról 0,6-ra emeltem, hogy ott erősebben jelezzem az ágensnek a helyzetet. Az előzés lépésjutalmát megnöveltem, 50 km/h-nál 0,90 helyett 1,10 járt, a tervem az volt, hogy az előzés többet érjen a követésnél. A lemaradás büntetését az előző 25 m helyett 40 m-re kiterjesztettem, ettől azt vártam, hogy a modell ne maradjon le messze a lassabb jármű mögött. A várt hatással ellentétben a modell tovább lassult. Átlagsebessége 31,7 km/h-ra esett, menetideje 3941 s-ra nőtt. A legnagyobb jelentősebben az autópályán volt érzékelhető, ugyanis itt sebesség 35,0 km/h-ról 29,5 km/h-ra csökkent. Az ágens gyakrabban állt meg ok nélkül az ez okozta visszahelyezések száma 3-ról 14-re nőtt (8. táblázat), a jelenség oka a megduplázott TTC büntetés és a felezett idő.

Az R4 esetében ami ugye a haladás alapú modell, Ha az ágens teszem fek 25 km/h-s forgalmi szereplő mögött haladt lépésenként 0,5 jár az R3 $-0{,}12$-je helyett, de ugye ez alatt értem a helyzeteket amikor kihagyta az előzést,. Ezek mellett amikor szabadott elkerülést végrehajtani akkor 50 km/h nál az 1,0-t kapott az R31,10-e helyett. A TTC-büntetés kikerült, az ütközést csak

a -30-as terminális büntetés tanítja. Összességében ezen a területen R4 hozta a legnagyobb javulást. Az előző R3-hoz képest a sebesség 31,7-ről 39,4 km/h-ra nőtt, ami gyakorlatilag az R1 39,6 km/h-s szintje, a menetidő pedig 3941-ről 3174 s-ra csökkent, ami az R1 3121 s-ához áll közel. Az R2-vel és az R3-mal ellentétben a modell már nem maradt a lassabb jármű mögött, hanem külön ösztönzés nélkül is előzött. A célba érések száma az R1-hez hasonlóan 49 lett.

Az R5 modellbe az R4 eddigi jutalmai mellé bekerült a potenciálalapú előzési jutalom került. A kihúzódás pillanatában a csak a manőver végén érkező $+5$ bónusz diszkontált értéke 0,66 volt, most viszont azonnal $+2$ jár, az elhaladás végére pedig a potenciál 5-re nő. Az R4-hez képest a sebesség 39,4-ről 40,1 km/h-ra nőtt, ez a különbség azonban nem szignifikáns. A végigjárási idő 3174-ről 3056 s-ra csökkent, ami az összes modell közül a legjobb, és mind az 50 bejárás célba ért.

Célba nem ért bejárások. Következőkben nézzük miért is nem teljesítették a modellek az egyes szakaszokat. Az ágensek vezetési képességének a kihívást jelentették a városból autópályára vezető útszakaszok. A 2 sikertelen bejárás a városból induló 2. és 4. számú pályákon történt.

Az R1 a 2. útvonal 2. körében 0,54 km után akadt el. Mind a 10 visszahelyezés ugyanazon a ponton történt. Az ágens egy kanyarban akadt el, a kormány 0,5–0,7-es kitérésben volt. Itt valamiért felgyűlt a modell hibája, és nem volt képes kezelni a szituációt. A másik kirívó eset a kiértékelések során az R4 modell okozta, mégpedig a 4. pálya 4. körében akadt el. Ezen esetben a 10 letérésből kettő 1,52 km-nél, nyolc a 3,16–3,35 km-es szakaszon. Itt található az autópálya-felvezető, S alakú felhajtó. Itt a modell egy előtte 2–3 m-re haladó jármű mögött újra és újra 2 m fokozatosan kisodródott a sáv közepéből, ami letérésnek számít, és visszahelyezés után néhány tíz méterrel később ugyanez megismétlődött. Ezen a szakaszon a többi modell legfeljebb egyszer tért le, majd célba ért.

6.3.2. Előzések sikerességének vizsgálata

Ebben a részben vizsgálom az ágensek egyik legfontosabb karakterisztikáját, azt, hogy hogyan is képesek elkerülni a forgalom többi résztvevőit. Ezeket a manővereket két szinten mértem ki. Először is az elhaladás minőségének szintjén minden jármű, amely mellett a modell elhaladt, vagy szabályos, vagy szabálytalan, értem ez alatt, hogy szépen húzódott mellé, nem ment túl közel az elkerülendő járművekhez, visszahúzódáskor nem zár rá a járműre. Ezeknek arányát adja meg a szabályos elhaladások százaléka. Másodlagosan arra voltam kíváncsi, hogy a megkezdett előzések hány százaléka zárult szabályosan, ütközés nélkül. Előfordultak esetek, amikor az ágens belekezdett egy előzési manőverbe de végül feladta, és visszasorolt a saját sávjába.

Az elhaladások számát és szabályosságát a 5. táblázat mutatja, az előzési kísérletek sikerességét pedig a 6. táblázat foglalja össze. Mindkét táblázatban útvonaltípusonként szerepelnek az értékek, az adott típus mind a 25 bejárását összesítve.

5. táblázat. Az elhaladások száma, ebből a szabályos és a szabálytalan elhaladások, valamint a szabályos elhaladások aránya útvonaltípusonként.

				Szabályos arány		
	Elhaladás	Szabályos	Szabálytalan	Autópálya	Városból autópályára	Összes
R1	1009	948	61	97,6 %	87,8 %	94,0 %
R2	827	772	55	96,9 %	86,7 %	93,3 %
R3	556	505	51	96,0 %	83,7 %	90,8 %
R4	1018	919	99	96,6 %	80,2 %	90,3 %
R5	1017	934	83	95,1 %	86,3 %	91,8 %

6. táblázat. A sikeres, vagyis szabályosan és ütközés nélkül zárult előzési kísérletek aránya útvonaltípusonként.

	Sikeres kísérlet		
	Autópálya	Városból autópályára	Összes
R1	78,3 %	70,2 %	74,6 %
R2	63,2 %	56,2 %	60,4 %
R3	45,4 %	46,4 %	45,8 %
R4	79,3 %	64,4 %	72,3 %
R5	75,4 %	74,4 %	75,0 %

A 5. táblázatból az látszik, már rögtön elsőre, hogy csak autópályán minden ágens szabályosabban előzött (95–98%), mint a városból induló útvonalakon (80–88%). Első ágensem, a kiindulási R1 modell összesen 1009 jármű mellett haladt el, ebből 948 volt szabályos elhaladás és 61 szabálytalan, ami 94,0%-os szabályos arányt jelent, ez az összes modell közül a legjobb, a megkezdett előzéseinek pedig 74,6%-a sikerült (6. táblázat).

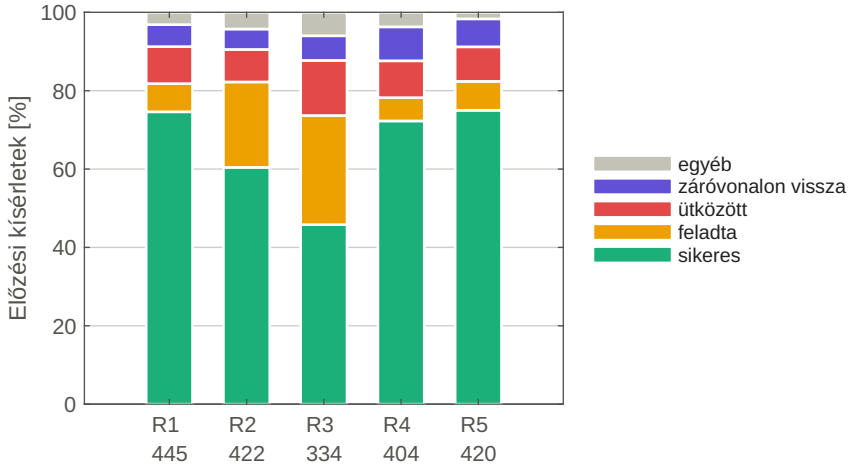
Az R2 modell az előzhető helyzetben járó TTC-büntetés 0-ról legfeljebb $-0{,}46$-ra nőtt a kiegészítés okán ez R1-ben nem járt itt viszont R2-ben már 7 s-os küszöbbel és 0,3-es súllyal adtam a modellnek. Ennek eredményeként az elhaladások száma 1009-ről 827-re csökkent (772 szabályos, 55 szabálytalan), a szabályos arány 94,0-ről 93,3%-ra, a sikeres kísérleteké 74,6-ról 60,4%-ra esett, autópályán 78,3-ról 63,2%-ra, a városból induló útvonalakon 70,2-ről 56,2%-ra. Itt jól látszik, hogy az ágens nem mert annyira előzni. Ezt a visszaesést hivatott orvosolni az R3 függvény ágense. Ebben a TTC-küszöb 7-ről 3 s-ra csökkent, a súlya 0,3-ról 0,6-ra, az előzés lépésjutalma 50 km/h-nál 0,90-ről 1,10-re nőtt, a lemaradás büntetése 25 helyett 40 m-ig tartott, a $+5$-ös bónusz pedig $+2$ és $+3$ részre vált. Ennek ellenére az elhaladások száma 827-ről 556-ra esett (505 szabályos, 51 szabálytalan), a szabályos arány 93,3-ról 90,8%-ra, a sikeres kísérleteké pedig 60,4-ről 45,8%-ra csökkent. Ennek oka az, hogy mivel a büntetés megduplázódott a

távolság felénél, így a modell még annyiszor se mert kiállni előzni.

Ezen a ponton a nagyobb áttörést az R4-től vártam vagyis a haladás utáni jutalomtól. Egy 25 km/h-s jármű mögött lépésenként 0,5 az R3 $-0{,}12$-je helyett, előzés közben 50 km/h-val 1,0 az 1,10 helyett, a TTC-büntetés pedig kikerült. Ezekkel a beállításokkal az ágens újra sokat előzött, valamint az elhaladások száma 556-ról 1018-ra, a sikeres kísérletek aránya 45,8-ról 72,3%-ra nőtt.

Itt az 1018 előzésből már csak 99 volt szabálytalan ennek köszönhetően az arány 90,8-ról 90,3%-ra kissé csökkent, a városból induló útvonalakon pedig 80,2% lett, ami ott a legalacsonyabb. Ennek oka meglátásom szerint, hogy R4-ben nincs követési jutalom, ami a lassabb jármű mögött tartaná az ágenst. Amikor nem szabad előzni, hiába van ott a folytonos vonal, az ágens akkor is belemegy az előzési manőverbe és megkísérli elkerülni az előtte haladót.

Az utolsó vagyis az R5 esetén kihúzódás pillanatában azonnal $+2$ jár, a manőver végén érkező $+5$ bónusz 0,66-os diszkontált értéke helyett. Ezeknek eredménye, hogy összesen 1017 elhaladásból 934 volt szabályos és 83 szabálytalan, a szabályos arány 90,3-ról 91,8%-ra növekedett. Ezen modellnél látható, hogy a sikeres kísérletek aránya 72,3-ról 75,0%-ra nőtt, ez a legmagasabb érték, bár az R1 74,6%-a szinte ugyanennyi. A kiértékelések során jobbról történő elhaladást, tehát ha az ágens saját helyén maradt és a bal sávban volt egy nála lassabb jármű nem számoltam szabálytalannak. A szabálytalan előzések főként olyan helyzetekből adódtak, amikor az ágens olyan helyen előzött, ahol az előzés nem volt megengedett, vagy záróvonalon át sorolt vissza. Ennek oka valószínűleg az autoenkóderek rekonstrukcióiban keresendő.



19. ábra. Az előzési kísérletek kimenetelének megoszlása, a modell neve alatt a kísérletek száma. Egyéb: visszahelyezés vagy a jelenet vége szakította félbe, vagy már megelőzött járművet előzött újra.

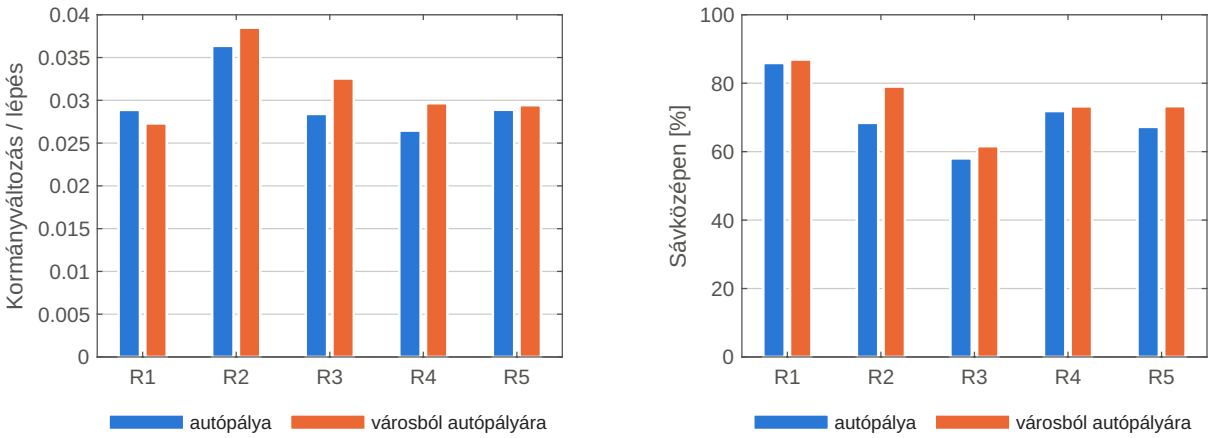
A megkezdett előzések kimenetelét a 19. ábra mutatja. Az egyéb kategóriába azok a kísérletek tartoznak, amelyeket egy visszahelyezés vagy a jelenet vége szakított félbe, illetve amikor a modell egy már korábban megelőzött járművet előzött újra. Az R1 a kísérleteinek csak 7,2%-át adta fel,

az R2 a kihúzódás után is járó TTC-büntetés miatt már 21,8%-át, az R3 pedig 27,8%-át, nála az ütközéssel végződő kísérletek aránya is 14,1%-ra nőtt. Az R4-ben, ahol előzés közben semmi nem büntet, a feladottak aránya 5,9%-ra esett, az R5-nél pedig alig változott (7,4%).

6.3.3. Kormányzás és sávtartás

7. táblázat. A kormányjel lépésenkénti átlagos abszolút változása és a sáv közepétől 0,5 m-en belül töltött idő aránya útvonaltípusonként.

	Kormányváltozás lépésenként			Sávközépen töltött idő		
	Autópálya	Városból autópályára	Összes	Autópálya	Városból autópályára	Összes
R1	0,029	0,027	0,028	85,9 %	86,9 %	86,4 %
R2	0,036	0,038	0,037	68,4 %	79,0 %	73,7 %
R3	0,028	0,033	0,030	58,0 %	61,6 %	59,8 %
R4	0,027	0,030	0,028	71,9 %	73,2 %	72,5 %
R5	0,029	0,029	0,029	67,2 %	73,3 %	70,3 %



20. ábra. Balra: a kormányjel lépésenkénti átlagos változása. Jobbra: a sáv közepén töltött idő aránya. Mindkettő útvonaltípusonként.

Következő szempont ami alapján a modelleket mértem az kormányzás minőségére vonatkozott. Az alábbi 7. táblázat és 20. ábra mutatja ezek változását

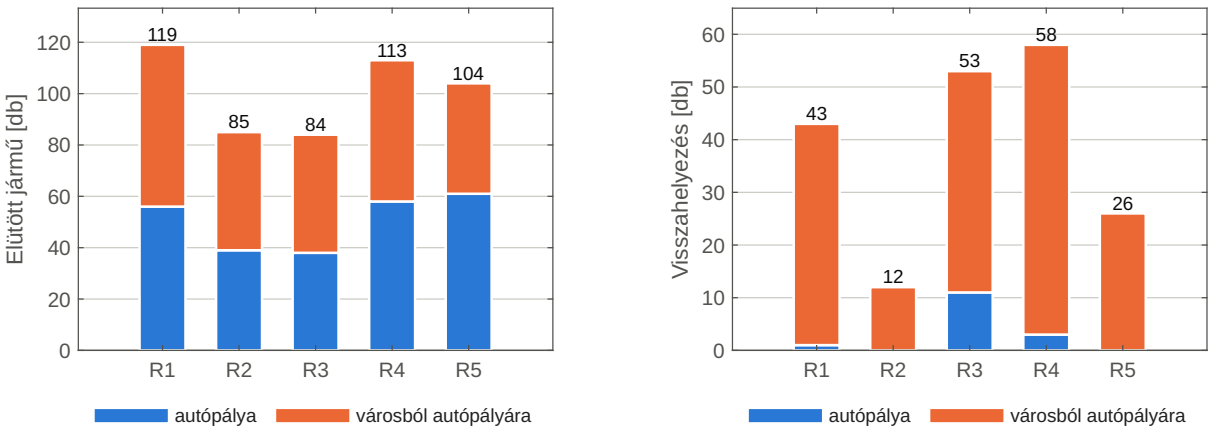
Az R1 modell esetében a sávtartásért két szorzó felelt ezek a szabad haladás valamint a követés jutalmában voltak beágyazva. Ezek a c sávközép-tényező, amely a sáv közepén 1, attól 2 m-re 0, valamint az s simasági tényező, amely a sávközéptől mért távolság 1,5 s alatti szórása alapján 0 m-nél 1, 0,35 m-nél 0. Emellett minden modellben közös a kormányrángatás büntetése, amely a kormányjel lépésenkénti változásának kétszerese, és a rossz sávért járó $-0{,}5$. Az R1 így kormányzott a legsimábban (0,028), és a sávot is ez tartotta a legjobban (86,4%).

Az R2-ben és az R3-ban a sávtartási tagokhoz nem nyúltam, csak a TTC- és az előzési tagokat változtattam, a sávtartás mégis 73,7, majd 59,8%-ra romlott, az R2-ben a kormányváltozás 0,037-re nőtt. Ezt a megkezdett, de be nem fejezett elkerülési kísérletek okozzák, mindegyik egy plusz kormánymozdulatot ad oda és vissza. Az R4-ben a c és az s szorzót elhagytam, helyettük minden sávban csak a célsáv közepétől mért távolságot büntető l tényező maradt, a kormányrángatás büntetése és a $-0{,}5$ változatlan. Ezzel a sávtartás 72,5%-ra javult, a kormányváltozás 0,028-ra változott. Az utolsó ágens az R5-ben pedig, ahol ezekhez már nem nyúltam, alig változott (70,3%, 0,029).

6.3.4. Ütközések és visszahelyezések

8. táblázat. Elütött járművek és visszahelyezések száma, mind az 50 bejárást összesítve. Az elütési arány az elütött járművek száma az összes elhaladáshoz képest.

	Elütött jármű	Elütési arány	Visszahelyezés	A visszahelyezés oka		
				Letért	Megállt	Túl gyors
R1	119	11,8 %	43	20	16	7
R2	85	10,3 %	12	9	3	0
R3	84	15,1 %	53	15	14	24
R4	113	11,1 %	58	51	5	2
R5	104	10,2 %	26	22	2	2



21. ábra. Balra: az elütött járművek száma, jobbra: a visszahelyezések száma, a két útvonaltípus összege, a teljes szám az oszlopok felett.

A vezető ágensek egyik, ha nem a legfontosabb összehasonlítása következik, ahol azt vizsgálom, mennyire is vezetnek biztonságosan a modellek. Erre vonatkozó eredményeket a 8. táblázat

és a 21. ábra mutatja. Itt főként arra voltam kíváncsi, hogy a modell hány járművet üt el, és ez az elhaladásainak hány százaléka. A modellek nagyjából ugyanannyi utat tettek meg (169–171 km), így a számokat egymással is össze lehet hasonlítani.

Az R1 119 járművet ütött el, ez az elhaladásainak 11,8%-a, és 43-szor kellett visszahelyezni. Az R2 vezetett a legóvatosabban. A kiterjesztett TTC-büntetés miatt lassabban ment, így csak 85 járművet ütött el (10,3%), visszahelyezni pedig csak 12-szer kellett. Az R3-nál bátrabb előzést vártam, de pont ez lett a legveszélyesebb ágens. Az elütési arány 15,1%-ra nőtt, 53-szor kellett visszahelyezni, és ez volt az egyetlen, amelyik rendszeresen 60 km/h fölé gyorsított (24-szer). Szerintem ezt a megemelt előzési lépésjutalom okozta.

Az R4-ből kikerült a TTC-büntetés. Több járművet ütött el (113), de sokkal többet is előzött, így az elütési aránya 11,1%-ra csökkent. Tehát nem előzött veszélyesebben, csak többet. Nála a letérés volt a gond, az 58 visszahelyezésből 51 emiatt történt. Az utolsó, vagyis az R5 elütési aránya lett a legkisebb (10,2%), nagyjából annyi, mint az R2-é, és csak 26-szor kellett visszahelyezni. Az R4-hez képest viszont csak a visszahelyezések száma (kilométerenként 0,13-mal kevesebb) és a kormányzás változott szignifikánsan, a sebesség, az előzések és az ütközések nem. Ez egyezik azzal, amit az elmélet mond, vagyis hogy a potenciálalapú formálás nem változtatja meg az optimális stratégiát [28]. A visszahelyezések szinte mind a városból induló útvonalakon voltak, autópályán az R3-at leszámítva egyik ágenst sem kellett háromnál többször visszahelyezni.

7. Összegzés

Munkám célja egy önvezető ágens fejlesztése a CARLA szimulátorban, amely csak a kamera és a LiDAR adatok alapján vezet valamint képes biztonságosan elkerülni a nála lassabb forgalmi résztvevőket. Az ágens típusának a Soft Actor-Critic algoritmust választottam. Tanítás során nem a nyers szenzoradatokat, hanem előre betanított autoenkóderekkel tömörített információt adtam a modellnek. A kameraképet egy konvolúciós enkóder, a LiDAR pontfelhőjéből kinyert adatokat egy gráfalapú enkóder segítségével helyeztem a látens térbe. Ezen adatokat egy összeadás alapú fúzió után hozzáfűzöm az autó többi jellemzőjéhez.

A munka nagyobbik felét a jutalomfüggvény megfelelő definiálása volt. Ennek okán öt változatot is készítettem, különböző paraméterezésekkel majd megvalósításokkal. A modellek tanításait azonos beállításokkal végeztem egyetlen különbség a jutalom függvény volt csak. Az első modell, az R1 a sávtartást és a szabályos előzést jutalmazza külön definiált tagokkal. Az R2-ben és az R3-ban ezt próbáltam tovább hangolni ezeket a tagokat a jobb eredmény elérése érdekében, első körben a TTC-büntetés kiterjesztésével, majd az előzési tagok erősítésével akartam maximalizálni a modell viselkedését. A negyedik modellnél új megközelítést alkalmaztam itt csak a haladás után adtam jutalmat, az R5 pedig ezt egészíti ki egy potenciálalapú előzési jutalommal.

A kiértékelést tíz előre definiált útvonalon végeztem mindegyiken ötször, összesen 250

bejárást végeztem le. Mérőszámoknak olyanokat választottam, melyek függetlenek a jutalomtól.

A kézi hangolás nem egészen hozták a tőlük elvárt eredményeket. Az R2 és az R3 ágensek lelassultak, kevesebbszer és gyengébb minőségben voltak képesek az elkerülésre. A harmadik modell esetén hiába erősítettem az előzésre, az pont az ellenkezőjét váltotta ki a tanulás során. A haladásalapú R4 az új megközelítés segítségével különösebb ösztönzés nélkül is visszahozta az eredeti R1 sebesség valamint előzési teljesítményt is.

Összességében egyértelmű legjobb modellt nem lehet kimondani, mivel a két legjobb modell más-más dologban jobb. Az R5 volt a leggyorsabb (40,1 km/h, 3056 s), és mind az 50 bejárás során célba ért. Biztonság szempontjából is ez teljesített a legjobban, ugyanis ütközés aránya a legalacsonyabb (10,2%). Az R1-nél jóval kevesebbszer kellett visszahelyezni (26-szor 43 helyett). Az R1 modell viszont szabályosabban előzött (94,0% a 91,8% helyett), és sokkal jobban tartotta a sávnak a közepét (86,4% a 70,3% helyett). Az előzési kísérletek sikerességében a kettő gyakorlatilag egyforma (74,6 és 75,0%).

Az eredményekből fény derül arra, hogyan lehet érdemes egy ágenst mozgó járművek elkerülésére tanítani. Nem érdemes minden helyzetre külön kézi tagot írni, egyrészt ezek a tagok könnyedén kiolthatják egymást így az ágens nem azt tanulja meg, amit szeretnénk, hanem a szabály megkerülését. Nem bizonyult hatékonynak az ötlet miszerint előzés közben büntetem a közeledést a második modell esetén világos, hogy ettől az ágens nem fog hatékonyabban előzni. Jobban működik, ha a jutalom egyszerűen a haladást díjazza, így a lassabb autó mögött maradás magától kevesebbet ér, és az ágens rájön, hogy előzni megéri. Az előzés megkezdését pedig érdemes rögtön jutalmazni, ez látszik az R5 modellen is ahol a potenciálalapú tag életbe lépett, az eljárás nem változtat azon, hogy mi a legjobb stratégia, csak hamarabb jelez az ágensnek. Következő továbbfejlesztési lehetőség az lenne, ha az R5 függvénybe visszakerülne sávtartás büntetése, így az ágens ugyanolyan gyorsan és sikeresen előzne, de szebben tartaná a sávokat. Mindenképp az ötödik függvény továbbfejlesztésének irányában indulok.

8. Köszönetnyilvánítás

Ezúton szeretném megköszönni témavezetőmnek, Mészáros Lászlónak, a szakmai iránymutatást, segítséget valamint a sok hasznos kritikát, ezek segítettek abban, hogy a dolgozat a megfelelő irányba fejlődjön.

Nyilatkozat mesterséges intelligencia eszközök használatáról

Ezúton nyilatkozom, hogy a szakdolgozat elkészítése során az alábbi mesterséges intelligencia alapú eszközt használtam.

MI-eszköz használata

Használtam mesterséges intelligencia alapú eszközt a szakdolgozat készítése során az alábbiak szerint:

MI eszköz neve és verziója	**Pontosan mire használtam**
Claude Opus 5.5	- Általános szövegrészek generálására - Egyszerűbb kódblokkok gyorsabb elkészítésére - Szakirodalom áttanulmányozására és értelmezésére

Kijelentem, hogy a szakdolgozat szakmai tartalmáért teljes felelősséget vállalok, a végleges tartalmat én állítottam össze, az MI által generált tartalmakat átnéztem, ellenőriztem és szükség esetén javítottam, valamint az MI-használatot valósághűen és teljes körűen feltüntettem.

Hivatkozások

[1] World Health Organization. *Global Status Report on Road Safety 2023*. Geneva: World Health Organization, 2023. ISBN: 978-92-4-008651-7. URL: https://www.who.int/publications/i/item/9789240086517 (elérés dátuma 2026. 09. 29.).

[2] Steven Macenski és tsai. „Robot Operating System 2: Design, Architecture, and Uses in the Wild". *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074.

[3] Shinpei Kato és tsai. „Autoware on Board: Enabling Autonomous Vehicles with Embedded Systems". *2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPS)*. 2018, 287–296. old. DOI: 10.1109/ICCPS.2018.00035.

[4] Alexey Dosovitskiy és tsai. „CARLA: An Open Urban Driving Simulator". *Proceedings of the 1st Annual Conference on Robot Learning*. 2017, 1–16. old.

[5] Wasinee Terapaptommakol és tsai. „Design of Obstacle Avoidance for Autonomous Vehicle Using Deep Q-Network and CARLA Simulator". *World Electric Vehicle Journal* 13.12 (2022), 239. old. DOI: 10.3390/wevj13120239.

[6] CARLA Team. *CARLA: Open-source Simulator for Autonomous Driving Research*. URL: https://carla.org (elérés dátuma 2026. 10. 01.).

[7] CARLA Team. *CARLA Simulator Documentation (0.9.16)*. URL: https://carla.readthedocs.io/en/0.9.16/ (elérés dátuma 2026. 10. 01.).

[8] Dean A. Pomerleau. „ALVINN: An Autonomous Land Vehicle in a Neural Network". *Advances in Neural Information Processing Systems 1 (NIPS)*. 1989, 305–313. old.

[9] Richard S. Sutton és Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2. kiad. Cambridge, MA: MIT Press, 2018. ISBN: 978-0-262-03924-6.

[10] Volodymyr Mnih és tsai. „Human-level Control through Deep Reinforcement Learning". *Nature* 518.7540 (2015), 529–533. old. DOI: 10.1038/nature14236.

[11] Timothy P. Lillicrap és tsai. „Continuous Control with Deep Reinforcement Learning". *4th International Conference on Learning Representations (ICLR)*. 2016.

[12] Tuomas Haarnoja és tsai. „Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor". *Proceedings of the 35th International Conference on Machine Learning (ICML)*. 80. köt. Proceedings of Machine Learning Research. 2018, 1861–1870. old.

[13] Python Software Foundation. *The Python Programming Language*. URL: https://www.python.org (elérés dátuma 2026. 10. 02.).

[14] Adam Paszke és tsai. „PyTorch: An Imperative Style, High-Performance Deep Learning Library”. *Advances in Neural Information Processing Systems 32 (NeurIPS)*. 2019, 8024–8035. old.

[15] Antonin Raffin, Jens Kober és Freek Stulp. „Smooth Exploration for Robotic Reinforcement Learning”. *Proceedings of the 5th Conference on Robot Learning (CoRL)*. 164. köt. Proceedings of Machine Learning Research. 2022, 1634–1644. old.

[16] Tuomas Haarnoja és tsai. „Soft Actor-Critic Algorithms and Applications”. *arXiv preprint arXiv:1812.05905* (2018).

[17] Antonin Raffin és tsai. „Stable-Baselines3: Reliable Reinforcement Learning Implementations”. *Journal of Machine Learning Research* 22.268 (2021), 1–8. old.

[18] Mark Towers és tsai. „Gymnasium: A Standard Interface for Reinforcement Learning Environments”. *arXiv preprint arXiv:2407.17032* (2024).

[19] Mariusz Bojarski és tsai. „End to End Learning for Self-Driving Cars”. *arXiv preprint arXiv:1604.07316* (2016).

[20] Kaiming He és tsai. „Deep Residual Learning for Image Recognition”. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, 770–778. old. DOI: 10.1109/CVPR.2016.90.

[21] Karen Simonyan és Andrew Zisserman. „Very Deep Convolutional Networks for Large-Scale Image Recognition”. *3rd International Conference on Learning Representations (ICLR)*. 2015.

[22] Jiuming Liu és tsai. „TopoLiDM: Topology-Aware LiDAR Diffusion Models for Interpretable and Realistic LiDAR Point Cloud Generation”. *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2025. arXiv: 2507.22454.

[23] Yatian Pang és tsai. „Masked Autoencoders for Point Cloud Self-supervised Learning”. *Computer Vision – ECCV 2022*. 2022, 604–621. old. DOI: 10.1007/978-3-031-20086-1_35. URL: https://www.ecva.net/papers/eccv_2022/papers_ECCV/papers/136620591.pdf.

[24] Quentin Garrido és tsai. „RankMe: Assessing the Downstream Performance of Pretrained Self-Supervised Representations by Their Rank”. *Proceedings of the 40th International Conference on Machine Learning (ICML)*. 202. köt. Proceedings of Machine Learning Research. 2023.

[25] Maxime Oquab, Timothée Darcet, Théo Moutakanni és tsai. „DINOv2: Learning Robust Visual Features without Supervision”. *Transactions on Machine Learning Research* (2024).

[26] Min Mei és A. G. R. Bullen. „Lognormal Distribution for High Traffic Flows”. *Transportation Research Record* 1398 (1993), 125–128. old.

[27] Meixin Zhu és tsai. „Safe, Efficient, and Comfortable Velocity Control based on Reinforcement Learning for Autonomous Driving”. *Transportation Research Part C: Emerging Technologies* 117 (2020), 102662. old. arXiv: 1902.00089.

[28] Andrew Y. Ng, Daishi Harada és Stuart Russell. „Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping”. *Proceedings of the 16th International Conference on Machine Learning (ICML)*. 1999, 278–287. old.